Test Case Document - Online Internship & Placement Preparation Portal

Project	Online Internship & Placement Preparation Portal (OIPPP)
Version	2.0
Authors	Deekshith Gowda G, Sharath H G, Guru R Bharadwaj, E S Sukruth
Date	07-11-2026
Replaces	Software Test Plan v1.0 (05-10-2026), section 13

1. Purpose

This document holds every test case for release 1.0 of OIPPP, updated after the product was built, and the requirements traceability matrix (RTM) that links each SRS requirement to its test cases and automated tests. The test cases were run against the Part-3 test build from 2 to 6 Nov 2026. Their results are summarised in the System Validation Report.

2. Version history

Version	Date	Author	Changes
1.0	05-10-2026	All members	16 test cases (12 functional, 4 non-functional) in Test Plan section 13, written before development
2.0	06-11-2026	All members	The 16 cases updated to match what was built (section 4), new cases for every requirement that v1.0 covered only in the automated suite, UI steps added, "Automated by" added for each case, the RTM updated (section 6), and actual results from the Part-3 test build filled in

3. How the test cases are written

Every case uses the fields of Test Plan v1.0 section 13, plus three:

Field	Meaning
Requirement(s)	SRS requirement IDs the case proves
Type	Functional, Negative, Boundary, Security, Performance, Reliability or UI
Preconditions, Steps, Test data	What must be true before the run, the API and UI steps, and the inputs
Expected result	Status codes, error codes and messages from the API contract, and what the screen shows
Actual result, Status	What happened on the Part-3 test build: Pass, Fail, Blocked or Not run, with the environment, date and commit
Automated by	The Jest or Vitest tests that check the same behaviour on every CI run
Changes from v1.0	What changed and the DEVIATIONS row behind it, or "New"

The existing IDs are kept. New cases continue each module's numbering, for example TC-APP-03 after TC-APP-02.

The test build, accounts and data set are in deployment.md section 6.

4. Test cases by module

Module	File	Owner	Requirements
Identity and security	test-cases/identity.md	Deekshith Gowda G (#83)	AUTH, PROF, SEC, DOM-2
Listings and search	test-cases/listings.md	Sharath H G (#87)	LST, SRCH, PERF
Applications, review and notifications	test-cases/applications.md	Guru R Bharadwaj (#91)	APP, REV, DOM-1
Aptitude and analytics	test-cases/aptitude-analytics.md	E S Sukruth (#95)	TST, ANL, SAFE, DOM-3

The PDF places the four files in this order after this section.

Changes to the v1.0 test cases

TC	Change in v2.0	DEVIATIONS row
TC-AUTH-01	Duplicate-email error code and message	18
TC-AUTH-02	Explicit password-rule cases (length, letter, digit)	
TC-AUTH-03	423 `ACCOUNT_LOCKED` with `retryAt`	18
TC-AUTH-04	Step 3 uses `GET /applications/:id/resume`	16
TC-PROF-01	A 2.1 MB PDF returns 413 `FILE_TOO_LARGE`; DOCX returns 400 `INVALID_FILE_TYPE`	23
TC-LST-01	Deadline closer timing; editing a closed listing returns 409 `LISTING_CLOSED`	
TC-SRCH-01	Query uses `search`, `sortBy` + `order` and `mode=REMOTE`; response is `{listings, pagination}`; counts from the `seed:demo` data	19
TC-APP-01	Response body and `ALREADY_APPLIED` details per the API contract; recruiter notification, stored resume and the 1,000-character note limit added	18
TC-APP-02	`NOT_ELIGIBLE` code and exact messages; a listing past its deadline counts as closed	
TC-REV-01	`INVALID_TRANSITION` and `REASON_REQUIRED` codes; Mailtrap email check; retries after 1, 5 and 15 minutes	1, 4
TC-TST-01	Acknowledged saves; same attempt and same `endsAt` after reopening	
TC-TST-02	Hard cutoff: any answer at or after `endsAt` returns 409 `ATTEMPT_CLOSED`; auto-submit by the timeout sweep	10, 11

TC	Change in v2.0	DEVIATIONS row
TC-ANL-01	`GET /analytics/placements` and `/analytics/placements/export`; no combined readiness score	13, 21
TC-TST-06	New: answer save acknowledged within 1 s; failed on Render (4 of 20 saves over 1 s, bug #118)	28
TC-PERF-01	`npm run test:load` or `search_load.jmx` against the deployed build; PERF-3 added	
TC-SEC-01	bcrypt cost 12, so the stored hash starts with `$2b$12$`	
TC-SEC-02	11th login returns 429 `RATE_LIMITED`; HTTPS and TLS checked on the deployed hostnames	

5. Summary of results

Status on 07-11-2026 (final), taken from the four module files and the System Validation Report section 4.1.

Module	Test cases	Pass	Partial	Fail	Blocked	Not run
Identity and security	14	13	1	0	0	0
Listings and search	11	8	2	0	1	0
Applications, review and notifications	9	9 (API; UI for 3)	0	0	0	0
Aptitude and analytics	17	15	1	1	0	0
Total	**51**	**45**	**4**	**1**	**1**	**0**

The one failure is TC-TST-06 (TST-4 save latency on the free Render instance, bug #118, DEVIATIONS row 28). TC-SEC-02 failed on 06-11 (bug #124), was fixed in PR #125 and passes on the 07-11 Render retest. TC-PERF-01 is partial: PERF-1 passes, PERF-2 fails on Render with p95 1,679 ms after the #123 fix (row 29). The five admin-only identity cases (PROF-3, PROF-4, DOM-2, SEC-6, TC-SEC-01) were run on Render on 07-11-2026 and pass. The one blocked case is PERF-3, which needs 2,000 students and passes on the local build.

Schedule deviation

System testing and UAT ran from 2 to 6 Nov 2026, not 9 to 23 Nov as planned in Test Plan section 7 (DEVIATIONS row 27).

6. Requirements traceability matrix

"v1.0" is the Test Plan section 13 RTM. "Automated by" lists the main suites under `server/tests/`; the module files name the individual tests.

Requirement	v1.0 test case(s)	v2.0 test case(s)	Automated by
AUTH-1	TC-AUTH-01	TC-AUTH-06	`integration/auth.test.js`
AUTH-2	TC-AUTH-01	TC-AUTH-01	`integration/auth.test.js`
AUTH-3	TC-AUTH-02	TC-AUTH-02	`integration/auth.test.js`, `unit/auth-validation.test.js`

Requirement	v1.0 test case(s)	v2.0 test case(s)	Automated by
AUTH-4	TC-AUTH-03	TC-AUTH-05	`integration/auth.test.js`, `unit/auth-service.test.js`
AUTH-5	TC-AUTH-03	TC-AUTH-03	`integration/auth.test.js`
AUTH-6	TC-AUTH-04	TC-AUTH-04	`integration/rbac.test.js`
PROF-1	Unit/integration suite only	PROF-1	`integration/profile.test.js`, `unit/profile-model.test.js`
PROF-2	TC-PROF-01	TC-PROF-01	`integration/resume.test.js`
PROF-3	Unit/integration suite only	PROF-3	`integration/admin.test.js`
PROF-4	Unit/integration suite only	PROF-4	`integration/identity-flow.test.js`, `integration/rbac.test.js`
LST-1	TC-LST-01	TC-LST-01, UI-LST-02	`integration/listings.test.js`
LST-2	TC-LST-01	TC-LST-01, UI-LST-02	`integration/listings.test.js`
LST-3	Unit/integration suite only	LST-3	`integration/listings.test.js`
LST-4	TC-LST-01	TC-LST-01	`integration/listings.test.js`
SRCH-1	TC-SRCH-01	TC-SRCH-01, UI-LST-01	`integration/listing-search.test.js`
SRCH-2	TC-SRCH-01	TC-SRCH-01, UI-LST-01	`integration/listing-search.test.js`
SRCH-3	TC-SRCH-01	TC-SRCH-01, UI-LST-01	`integration/listing-search.test.js`
SRCH-4	Unit/integration suite only	SRCH-4, UI-SRCH-04	`integration/bookmarks.test.js`
SRCH-5	Unit/integration suite only	SRCH-5, UI-SRCH-05	`integration/listing-recommendations.test.js`
APP-1	TC-APP-01	TC-APP-01	`integration/apply.test.js`, `integration/reliability.test.js`
APP-2	TC-APP-02	TC-APP-02	`integration/apply.test.js`
APP-3	TC-APP-02	TC-APP-02, TC-APP-04	`integration/apply.test.js`
APP-4	TC-APP-01	TC-APP-01, TC-REV-03, TC-REV-04	`integration/apply.test.js`, `integration/status.test.js`, `integration/audit.test.js`
APP-5	Unit/integration suite only	TC-APP-03	`integration/apply.test.js`
REV-1	Unit/integration suite only	TC-REV-02	`integration/review.test.js`, `integration/resume.test.js`
REV-2	TC-REV-01	TC-REV-01, TC-REV-03, TC-REV-04	`integration/status.test.js`, `unit/application-transitions.test.js`

Requirement	v1.0 test case(s)	v2.0 test case(s)	Automated by
REV-3	TC-REV-01	TC-REV-01, TC-REV-03	`integration/review.test.js`
REV-4	TC-REV-01	TC-REV-01, TC-REV-05	`integration/notifications.test.js`, `integration/status.test.js`
TST-1	Unit/integration suite only	TC-TST-03, UI-TST-04	`integration/aptitude-api.test.js`, `unit/aptitude-models.test.js`
TST-2	Unit/integration suite only	TC-TST-04, UI-TST-04	`integration/aptitude-api.test.js`
TST-3	TC-TST-01	TC-TST-01, TC-TST-05, UI-TST-01	`integration/aptitude-recovery.test.js`
TST-4	TC-TST-01	TC-TST-01, TC-TST-06, TC-SAFE-01, UI-TST-02	`integration/aptitude-recovery.test.js`, `integration/aptitude-indexes.test.js`
TST-5	TC-TST-02	TC-TST-02, TC-SAFE-01, UI-TST-02	`integration/aptitude-recovery.test.js`, `unit/attempt-timeout-job.test.js`
TST-6	TC-TST-02	TC-TST-02, TC-TST-05, UI-TST-03	`unit/aptitude-scoring.test.js`
ANL-1	Unit/integration suite only	TC-ANL-02, UI-ANL-01	`integration/analytics.test.js`
ANL-2	TC-ANL-01	TC-ANL-01, TC-DOM-03, UI-ANL-02	`integration/analytics.test.js`
ANL-3	TC-ANL-01	TC-ANL-01, UI-ANL-02	`integration/analytics.test.js`
PERF-1	TC-PERF-01	TC-PERF-01	`integration/listing-search-performance.test.js`; load run (#89)
PERF-2	TC-PERF-01	TC-PERF-01	Load run (#89)
PERF-3	Reviewed in Sprint 3	PERF-3, UI-ANL-01, UI-ANL-02	`integration/analytics-performance.test.js`; load run (#89)
SAFE-1	TC-TST-01	TC-TST-01, TC-SAFE-01, UI-TST-02	`integration/aptitude-recovery.test.js`
SAFE-2	Reviewed in Sprint 3	TC-SAFE-02	`integration/identity-edge-cases.test.js`
SEC-1	TC-SEC-01	TC-SEC-01	`unit/auth-service.test.js`
SEC-2	TC-SEC-02	TC-SEC-02 (deployed-URL part, #85)	Deployed-URL check (#85)
SEC-3	TC-AUTH-04	TC-AUTH-04	`integration/rbac.test.js`, `integration/resume.test.js`
SEC-4	TC-SEC-01	TC-SEC-01	`unit/validate-middleware.test.js`
SEC-5	TC-SEC-02	TC-SEC-02	`integration/auth.test.js`

Requirement	v1.0 test case(s)	v2.0 test case(s)	Automated by
SEC-6	Security checklist 5.1	SEC-6	`integration/audit.test.js`, `integration/status.test.js`
DOM-1	Not listed	TC-APP-02, TC-APP-04	`integration/apply.test.js`
DOM-2	Not listed	DOM-2	`integration/settings.test.js`
DOM-3	Not listed	TC-DOM-03, UI-ANL-02	`integration/analytics.test.js`

Quality attributes QA-1 to QA-5 are measured in the System Validation Report (UAT and devices in #97, QA-3 and QA-4 from the automated suites), not by individual test cases.

Identity and Security Test Cases

Project: OIPPP **Version:** Part-3 Testing **Environment:** Local / Render (where applicable) **Date:** 3 November 2026 **Issue:** #83 – Update identity & security test cases

1. Purpose

This document contains the updated identity, authentication, profile and security test cases for OIPPP.

The test cases cover the requirements listed in `docs/other-docs/identity-security.md`.

Actual results and execution status must be updated only after the tests are performed.

2. Updated Existing Test Cases

TC-AUTH-01: Duplicate Email Registration

Field	Value
Requirement(s)	AUTH-2
Type	Negative
Preconditions	An account already exists with the email address being tested.
Steps	1. Open registration. 2. Enter an already registered email. 3. Submit the registration form.
Test data	Existing registered email address.
Expected result	HTTP 409 with error code `DUPLICATE` and message `An account with this email already exists.`
Actual result	**Render, 06-11-2026:** registering `student0001@students.example.com` again returned 409 `DUPLICATE`, "An account with this email already exists". Evidence: `docs/testing/evidence/identity-render-results.json`.
Status	Pass
Automated by	`server/tests/integration/auth.test.js` – duplicate registration test.
Changes from v1.0	Updated to specify HTTP 409, `DUPLICATE`, and the exact error message.

TC-AUTH-02: Password Validation

Field	Value
Requirement(s)	AUTH-3
Type	Negative
Preconditions	Registration page is available.
Steps	1. Open registration. 2. Enter a password shorter than 8 characters. 3. Try a password without a letter. 4. Try a password without a digit. 5. Submit each request.
Test data	`abc`, `abcdefgh`, `12345678`

Field	Value
Expected result	Each invalid password is rejected with HTTP 400 and an appropriate validation error.
Actual result	**Render, 06-11-2026:** `abc` → 400 `VALIDATION` "length must be at least 8 characters long"; `abcdefgh` → 400 "Password must contain at least one digit"; `12345678` → 400 "Password must contain at least one letter". Evidence: `docs/testing/evidence/identity-render-results.json`.
Status	Pass
Automated by	`server/tests/integration/auth.test.js` and `server/tests/unit/auth-validation.test.js`.
Changes from v1.0	Added explicit password validation scenarios.

TC-AUTH-03: Account Lockout After Failed Login Attempts

Field	Value
Requirement(s)	AUTH-5
Type	Security / Negative
Preconditions	An active user account exists.
Steps	1. Submit five incorrect passwords. 2. Attempt another login. 3. Check the response and retry timestamp.
Test data	Valid account email and five incorrect passwords.
Expected result	The account is locked for 15 minutes. The next login returns HTTP 423 with `ACCOUNT_LOCKED` and a `retryAt` field.
Actual result	**Render, 06-11-2026:** on a fresh student account, wrong passwords 1–5 returned 401; the 6th attempt returned 423 `ACCOUNT_LOCKED` with `retryAt` 18:30:11Z (15 minutes after the 5th failure), and the correct password was also refused with 423 while locked. After `retryAt` (18:30:33Z) the correct password logged in with 200. Evidence: `docs/testing/evidence/identity-render-results.json`, `identity-render-results-3.json`.
Status	Pass
Automated by	`server/tests/integration/auth.test.js` and `server/tests/unit/auth-service.test.js`.
Changes from v1.0	Updated to specify HTTP 423, `ACCOUNT_LOCKED`, and `retryAt`.

TC-AUTH-04: Resume Download Authorization

Field	Value
Requirement(s)	AUTH-6, SEC-3
Type	Security / Negative
Preconditions	A student who owns a resume, a recruiter who has an application for that resume, an admin, and an unrelated user exist.
Steps	1. Authenticate as an unrelated user and request `GET /applications/:id/resume`. 2. Repeat as the student who owns the resume. 3. Repeat as a recruiter associated with

Field	Value
	the relevant application. 4. Repeat as an authorized admin.
Test data	Application ID, resume belonging to another user, and authorized user accounts with different roles.
Expected result	Unrelated users receive HTTP 403. The owner, permitted recruiter and authorized admin can access the resume according to the access policy.
Actual result	**Render, 06-11-2026:** on an application by `student0003`: another student 403 `FORBIDDEN` "You cannot access this application"; another recruiter 403 `NOT_LISTING_OWNER`; no token 401. On an application sent with a resume (`6aee18d7…`), the owning student and the listing's recruiter both got 200 `application/pdf` (20,016 bytes). The admin path was not run (admin password not available to the tester). Evidence: `docs/testing/evidence/identity-render-results-2.json`, `identity-render-results-3.json`.
Status	Pass for student, recruiter and unrelated users; admin path not run
Automated by	`server/tests/integration/rbac.test.js`, `server/tests/integration/resume.test.js`, and `server/tests/integration/status.test.js`.
Changes from v1.0	Updated to identify `GET /applications/:id/resume` and cover the permitted access roles.

TC-PROF-01: Resume Upload Validation

Field	Value
Requirement(s)	PROF-2
Type	Boundary / Negative
Preconditions	An authenticated student account exists.
Steps	1. Upload a valid PDF. 2. Upload a PDF larger than 2 MB. 3. Upload a DOCX file.
Test data	Valid PDF, 2.1 MB PDF, and DOCX file.
Expected result	Valid PDF is accepted. A 2.1 MB PDF returns HTTP 413 `FILE_TOO_LARGE`. A DOCX file returns HTTP 400 `INVALID_FILE_TYPE`.
Actual result	**Render, 06-11-2026 (new student account):** a 20 KB PDF returned 200 "Resume uploaded successfully"; a 2.1 MB PDF 413 `FILE_TOO_LARGE` "Resume must not exceed 2 MB"; a `.docx` 400 `INVALID_FILE_TYPE` "Only PDF resumes are allowed". Evidence: `docs/testing/evidence/identity-render-results.json`.
Status	Pass
Automated by	`server/tests/integration/resume.test.js`.
Changes from v1.0	Updated to distinguish the 413 size error from the 400 file-type error.

TC-SEC-01: Password Hashing Using bcrypt

Field	Value
Requirement(s)	SEC-1

Field	Value
Type	Security
Preconditions	User registration or password creation is available.
Steps	1. Create a user account. 2. Inspect the stored password hash in the test environment. 3. Verify the bcrypt cost.
Test data	Test user password.
Expected result	Password is stored as a bcrypt hash using cost 12. The hash begins with `$2b$12$`. Plain-text passwords must not be stored.
Actual result	**Render, 07-11-2026:** new test account's stored hash starts `$2b$12$`, is 60 characters and does not contain the plain password. Evidence: `docs/testing/evidence/admin-render-results.json` (script `admin-render-run.mjs`).
Status	Pass
Automated by	`server/tests/unit/auth-service.test.js` and `server/tests/unit/validate-middleware.test.js`; cost configuration is also documented in `auth.service.js`.
Changes from v1.0	Updated to specify bcrypt cost 12 and the expected hash prefix.

TC-SEC-02: Login Rate Limiting

Field	Value
Requirement(s)	SEC-5
Type	Security / Boundary
Preconditions	Login endpoint is available and rate limiting is enabled.
Steps	1. Send ten login requests within the configured rate-limit window. 2. Send an eleventh request. 3. Inspect the response.
Test data	Eleven login requests from the same IP address.
Expected result	The first ten requests are processed according to authentication rules. The eleventh request returns HTTP 429 `RATE_LIMITED`.
Actual result	**Render, 06-11-2026: 11 failed logins in under a minute from one machine all returned 401; no 429.** A rerun with `curl` showed `ratelimit-remaining` going 9, 8, 7, 8, 8, 7, 7, 5, 6, 6, 5, 5, i.e. the requests were counted in several separate buckets, so the limit of 10 was never reached. The likely cause is the client IP the API sees behind Render's proxy changing between requests (`TRUST_PROXY=1`); not confirmed. The SVR 4.6 run on 02-11-2026 did get 429 on the 11th login. The limiter passes in the automated suite. Evidence: `docs/testing/evidence/identity-render-results.json`, `docs/testing/evidence/sec02-rerun.txt`. **Retest, Render 07-11-2026, after PR #125 (limiter keyed on the email): the 11th bad login returned 429 `RATE_LIMITED`.**
Status	Pass (Render retest 07-11-2026; failed on 06-11, bug #124, fixed)
Automated by	`server/tests/integration/auth.test.js` – authentication rate-limit test.
Changes from v1.0	Updated to specify the eleventh request and HTTP 429 `RATE_LIMITED`.

3. New Test Cases

PROF-1: Student Profile Editing

Field	Value
Requirement(s)	PROF-1
Type	Functional
Preconditions	A student account is registered and authenticated.
Steps	1. Log in as a student. 2. Open the profile page. 3. Edit branch, CGPA, graduation year, and skills. 4. Save the changes.
Test data	Valid branch, CGPA between 0 and 10, graduation year, and skills.
Expected result	Profile changes are saved and displayed correctly. CGPA must remain within the allowed range.
Actual result	Changes saved and persisted on the deployed UI (SVR section 4.3).
Status	Pass (deployed UI)
Automated by	Automation mapping to be confirmed from the relevant profile test files.
Changes from v1.0	New test case for student profile editing.

PROF-3: Admin Recruiter Approval and Rejection

Field	Value
Requirement(s)	PROF-3
Type	Functional / Security
Preconditions	An admin account and a pending recruiter account exist.
Steps	1. Log in as admin. 2. Open recruiter management. 3. Approve one pending recruiter. 4. Reject another pending recruiter with a valid reason code. 5. Inspect notifications and audit records.
Test data	Pending recruiters and valid rejection reason codes.
Expected result	Approval and rejection update recruiter status. Rejection requires a reason. Both decisions are audit-logged and generate in-app and email notifications.
Actual result	**Render, 07-11-2026:** two new recruiters registered as PENDING. Reject without a reason 400; approve 200 → ACTIVE; reject with `INVALID_COMPANY` 200 → REJECTED; 2 audit entries written. Evidence: `docs/testing/evidence/admin-render-results.json` (script `admin-render-run.mjs`).
Status	Pass
Automated by	`server/tests/integration/admin.test.js` – recruiter approval/rejection tests.
Changes from v1.0	New test case for admin recruiter decisions with reason codes.

PROF-4: Deactivated Account Token Revocation

Field	Value
Requirement(s)	PROF-4
Type	Security / Negative
Preconditions	A user account has a valid issued authentication token.
Steps	1. Log in and obtain a token. 2. Deactivate the account. 3. Use the previously issued token to access a protected endpoint.
Test data	Previously valid authentication token belonging to the deactivated account.
Expected result	The previously issued token is rejected because the account is deactivated.
Actual result	**Render, 07-11-2026:** new test student logged in (`/auth/me` 200); admin deactivated it (200, DISABLED); the same token then got 401 `UNAUTHENTICATED`; a fresh login was refused (403). Evidence: `docs/testing/evidence/admin-render-results.json` (script `admin-render-run.mjs`).
Status	Pass
Automated by	Automation mapping to be confirmed from the account-status integration tests.
Changes from v1.0	New test case for token revocation after account deactivation.

DOM-2: One-Offer Policy

Field	Value
Requirement(s)	Offer policy / one-offer restriction
Type	Functional / Negative
Preconditions	A student already holds an accepted offer.
Steps	1. Authenticate as the student. 2. Attempt to accept another offer. 3. Inspect the API response.
Test data	Student account holding one accepted offer and another offer to accept.
Expected result	The second offer acceptance is rejected with HTTP 422 `OFFER_HELD`.
Actual result	**Render, 06-11-2026:** `recruiter01` moved `student0003`'s application to INTERVIEW then OFFER (200 both); `student0003` then applied to another listing and got **201**, not 422. This matches the code: the one-offer policy is an admin setting that defaults to off (`settings.service.js`), and the deployed setting could not be switched on without the admin account. **Rerun 07-11-2026 with admin:** policy switched on, `student0003` (holds the OFFER) applied again and got **422 `OFFER_HELD`**; policy switched back off. Evidence: `identity-render-results-2.json`, `admin-render-results.json`.
Status	Pass
Automated by	API run recorded in the test report; matching Postman collection request.
Changes from v1.0	New test case for the one-offer policy.

SEC-6: Audit Log Integrity

Field	Value
Requirement(s)	SEC-6
Type	Security
Preconditions	Admin account and audit records exist.
Steps	1. Perform an admin account action or status change. 2. Inspect the generated audit entry. 3. Attempt to edit or delete the audit entry through available application routes.
Test data	Existing audit record and an admin action.
Expected result	Audit entries are created for applicable actions. Application routes do not allow audit entries to be edited or deleted.
Actual result	**Render, 07-11-2026:** the deactivation wrote a `USER_DEACTIVATED` audit entry; PATCH/DELETE probes on audit paths all return 404 (no route edits or deletes entries); the model refuses an update. Evidence: `docs/testing/evidence/admin-render-results.json` (script `admin-render-run.mjs`).
Status	Pass
Automated by	Automation mapping to be confirmed from audit-related tests.
Changes from v1.0	New test case for audit-log integrity.

4. UI Test Steps

4.1 Student Registration

1. Open the registration page.
2. Select the student role.
3. Enter name, email, password, and a valid SRN.
4. Submit the form.
5. Verify successful registration.
6. Try registering again with the same email.
7. Verify the duplicate-email error.

4.2 Login

1. Open the login page.
2. Enter valid credentials.
3. Verify successful login and redirection.
4. Log out.
5. Enter an incorrect password and verify the error response.
6. Perform the configured failed-login attempts and check the account-lockout behavior.

4.3 Profile Management

1. Log in as a student.
2. Open the profile page.
3. Edit branch, CGPA, graduation year, and skills.
4. Save the profile.
5. Verify that the updated information is displayed.
6. Test resume upload with valid and invalid files.

4.4 Admin Screens

1. Log in using an authorized admin account.
2. Open recruiter management.
3. View pending recruiter accounts.
4. Approve a pending recruiter.
5. Reject another pending recruiter with a valid reason.
6. Verify status changes and available audit/notification information.
7. Verify that a non-admin user cannot access admin-only screens.

5. Additional Automated Test Case – TC-AUTH-05

TC-AUTH-05: Verify JWT Expiration

Field	Value
Requirement(s)	AUTH-4
Type	Unit
Preconditions	JWT configuration is set to 60 minutes.
Steps	1. Log in using valid user credentials through the authentication service. 2. Verify the returned JWT using the configured JWT secret. 3. Check the token's `iat` and `exp` values. 4. Verify that the difference between `exp` and `iat` is 60 minutes.
Test data	Valid user login credentials.
Expected result	The JWT should expire exactly 60 minutes after it is issued.
Actual result	The automated test verified the 60-minute expiration successfully.
Status	PASS (local unit test)
Automated by	`server/tests/unit/auth-service.test.js`
Changes from v1.0	Added explicit verification of the 60-minute JWT expiration.

TC-AUTH-06: Successful Student Registration

Field	Value
Requirement(s)	AUTH-1
Type	Positive / Functional
Preconditions	The student registration page is available. The email and SRN are not already registered.
Steps	1. Open registration. 2. Select the student role. 3. Enter valid name, email, password, and SRN. 4. Submit the registration form.
Test data	Valid student details with a unique email and SRN.
Expected result	Registration succeeds and the student account is created.
Actual result	**Render, 06-11-2026:** API registration of a new student returned 201 with role STUDENT, status ACTIVE and the SRN. **UI (Playwright/Edge):** the Register form with name, institutional email, SRN and password created the account and opened `/home`

Field	Value
	("Welcome, TC"). Evidence: `docs/testing/evidence/identity-render-results.json`, `ui-register-results.json`, `ui-render/auth06-registered.png`.
Status	Pass (API and UI)
Automated by	UI manual test; automated mapping to be confirmed.
Changes from v1.0	Added a separate positive registration test case.

6. Requirement-to-Test Case Mapping

Requirement ID	Test Case
AUTH-1	TC-AUTH-06 – Successful student registration
AUTH-2	TC-AUTH-01 – Duplicate email registration
AUTH-3	TC-AUTH-02 – Password validation
AUTH-4	TC-AUTH-05 – JWT expires after 60 minutes (`server/tests/unit/auth-service.test.js`)
AUTH-5	TC-AUTH-03 – Account lockout
AUTH-6	TC-AUTH-04 – Role-based resume authorization
PROF-1	`server/tests/integration/profile.test.js`, `server/tests/unit/profile-model.test.js`
PROF-2	TC-PROF-01 – Resume upload validation
PROF-3	`server/tests/integration/admin.test.js`
PROF-4	`server/tests/integration/identity-edge-cases.test.js`
DOM-2	DOM-2 – One-offer policy
SEC-1	TC-SEC-01 – Password hashing using bcrypt
SEC-2	Deployment HTTPS verification – pending live execution
SEC-3	TC-AUTH-04 – Resume download authorization
SEC-4	`server/tests/integration/identity-flow.test.js`, `server/tests/integration/reliability.test.js`, `server/tests/unit/validate-middleware.test.js`
SEC-5	TC-SEC-02 – Login rate limiting
SEC-6	`server/tests/integration/audit.test.js`

7. Execution Notes

MongoDB Test Retry

- **Test file:** `tests/unit/user-profile-schema.test.js`
- **Environment:** Local Windows development environment.
- **Execution date:** 3 November 2026.
- **Command:** `npm test -- --runInBand tests/unit/user-profile-schema.test.js`
- **Result:** PASS.
- **Test suites:** 1 passed.

- **Individual tests:** 17 passed, 0 failed.
- **Execution time:** 9.64 seconds.
- **Observation:** The test suite passed independently after the initial full-suite run encountered a MongoMemoryServer startup timeout.

8. Changes Summary

- Updated the six requested existing identity and security test cases.
- Retained the existing password-validation test case.
- Added the five requested new test cases: PROF-1, PROF-3, PROF-4, DOM-2, and SEC-6.
- Added TC-AUTH-05 for JWT expiration verification.
- Added TC-AUTH-06 for successful student registration.
- Added UI testing steps for registration, login, profile management, and admin screens.
- Updated requirement-to-test-case mapping.
- Added automation references based on `docs/testing/test-report.md`, Section 3.
- 06-11-2026: executed the remaining cases against the Render build (API scripts and Playwright/Edge UI runs in `docs/testing/evidence/identity-render-run*.mjs`, `ui-register-run.mjs`). TC-SEC-02 failed on Render (no 429).
- 07-11-2026: after the admin password was reset, PROF-3, PROF-4, DOM-2, SEC-6 and TC-SEC-01 were run on Render and pass (`evidence/admin-render-run.mjs`). TC-SEC-02 retested on Render after the #124 fix and passes.

Listings and Search Test Cases

Project: OIPPP
Version: Part-3 Testing
Environment: Local / Render (where applicable)
Date: 4 November 2026
Issue: #87 – Update listings & search test cases

1. Purpose

This document records the functional, validation, authorization, search, saved-internship, recommendation and performance test cases for OIPPP listings.

It covers the listing and search requirements (`LST-1`–`LST-4`, `SRCH-1`–`SRCH-5`) and performance requirements (`PERF-1`–`PERF-3`).

Actual results and execution status must be updated only after the corresponding tests have been performed. Results from earlier release reports are identified as historical evidence and are not treated as a new execution on the Issue #87 branch.

2. Updated Existing Test Cases

TC-LST-01: Listing Creation, Validation and Expiry

Field	Value
Requirement(s)	LST-1, LST-2, LST-4
Type	Functional / Boundary / Negative
Preconditions	An approved recruiter account exists and is authenticated.
Steps	1. Create a listing with valid required fields and a future deadline. 2. Submit a listing with a past deadline. 3. Submit a listing with a negative stipend. 4. Create a listing with a short future deadline and wait for the expiry job. 5. Attempt to edit the expired/closed listing.
Test data	Valid company, title, location, mode, stipend, skills and future deadline; past deadline; negative stipend; short future deadline.
Expected result	A valid listing is created with status `OPEN`. A past deadline and negative stipend are rejected with HTTP 400 and validation errors. An expired listing is automatically changed to `CLOSED`; editing a closed listing returns HTTP 409 `LISTING_CLOSED`.
Actual result	**Render, 04-11-2026 (#88, SVR 4.4):** valid creation 201 `OPEN`; past deadline and negative stipend 400; a short-deadline listing later read `CLOSED` and editing it gave 409 `LISTING_CLOSED`. The exact expiry-job closing time was not measured.
Status	Pass
Automated by	`server/tests/integration/listings.test.js` – approved recruiter creation, past deadline, negative stipend, automatic expiry and closed-listing behavior.
Changes from v1.0	Clarifies the creation, validation and automatic-expiry scenarios and records the expected status transitions.

3. New Test Cases

LST-3: Listing Ownership and Recruiter Authorization

Field	Value
Requirement(s)	LST-3
Type	Authorization / Negative
Preconditions	Two approved recruiter accounts exist. Recruiter A owns an open listing. Recruiter B is authenticated separately.
Steps	1. Authenticate as Recruiter B. 2. Attempt to edit Recruiter A's listing. 3. Attempt to close Recruiter A's listing. 4. Authenticate as Recruiter A. 5. Edit the listing and close it.
Test data	Listing ID owned by Recruiter A; valid updated title and stipend; two different recruiter accounts.
Expected result	Recruiter B cannot edit or close Recruiter A's listing and receives HTTP 403 with the ownership error. Recruiter A can edit and close the listing successfully. The listing remains associated with its original recruiter.
Actual result	**Render, 04-11-2026 (#88, SVR 4.4):** the other recruiter got 403 for edit and close; the owner edited and closed the listing and GET confirmed `CLOSED`. **UI, 06-11-2026:** `recruiter02` opening `recruiter01`'s edit page got "You can only manage your own listings" (`evidence/ui-render/lst02-other-recruiter.png`).
Status	Pass
Automated by	`server/tests/integration/listings.test.js` – "only the owner can close a listing", "owner can close a listing", "owner can edit a listing" and "another recruiter cannot edit a listing".
Changes from v1.0	Adds explicit owner-versus-other-recruiter authorization scenarios for listing updates and closure.

SRCH-4: Saved Internships and Bookmark Limit

Field	Value
Requirement(s)	SRCH-4
Type	Functional / Boundary / Authorization
Preconditions	An authenticated student exists. At least one open, unexpired listing is available.
Steps	1. Save an open listing. 2. Save the same listing again. 3. Retrieve saved internships with `GET /api/bookmarks/mine`. 4. Remove the saved listing. 5. Attempt to save a closed listing and an expired listing. 6. Save 100 distinct eligible listings and attempt to save one additional listing. 7. Repeat a save for a listing already saved while at the limit.
Test data	Student account; open, closed and expired listings; 101 distinct eligible listings for the limit scenario.
Expected result	The first save returns HTTP 201. Repeating the save does not create a duplicate. `GET /api/bookmarks/mine` returns `{items}` with the student's populated listings, newest first. Removal returns HTTP 204. Closed and expired listings cannot be saved. The 101st distinct bookmark returns HTTP 422 with `BOOKMARK_LIMIT`. Re-saving an existing bookmark at the limit remains successful without increasing the count.

Field	Value
Actual result	**Render, 04-11-2026 (#88, SVR 4.4):** save 201; duplicate save returned the same bookmark ID; GET showed one occurrence; removal 204; closed and expired listings 400 `LISTING_NOT_BOOKMARKABLE`. The 100/101 boundary and re-save at the limit were not run on the shared data (covered by `integration/bookmarks.test.js`).
Status	Partial (limit boundary not run on Render)
Automated by	`server/tests/integration/bookmarks.test.js` – save, duplicate prevention, list, remove, role restriction, closed/expired listing rejection and 100-bookmark limit.
Changes from v1.0	Adds the saved-internship limit, duplicate-save behavior and eligibility checks.

SRCH-5: Student Listing Recommendations

Field	Value
Requirement(s)	SRCH-5
Type	Functional / Authorization
Preconditions	An authenticated student has a profile with skills and CGPA. Open listings with different skill matches and CGPA requirements exist.
Steps	1. Authenticate as the student. 2. Request `GET /api/listings/recommendations`. 3. Inspect the returned listings, `matchedSkills` and `matchCount`. 4. Verify ordering by match count. 5. Verify that closed, expired and above-CGPA listings are excluded. 6. Repeat with a student profile that has no skills.
Test data	Student profile with skills such as Node.js and MongoDB; eligible and ineligible listings; student profile with no skills.
Expected result	Eligible listings with matching skills are returned. Matches are case-insensitive and ranked by match count descending, with deadline used as the next ordering criterion. Each result includes its matched skills and match count. Closed, expired, above-CGPA and zero-match listings are excluded. A student with no skills receives an empty list. Unauthenticated requests return HTTP 401 and recruiter requests return HTTP 403.
Actual result	**Render, 04-11-2026 (#88, SVR 4.4):** 1,455 recommendations, all OPEN and unexpired, match counts consistent, ranking descending; no token 401, recruiter 403. **06-11-2026:** a new student with no skills got `{"listings":[]}` from `GET /listings/recommendations` (`evidence/srch05-empty-profile.json`) and the home page showed "No matches yet".
Status	Pass
Automated by	`server/tests/integration/listing-recommendations.test.js` – skill ranking, eligibility exclusions, empty profile and role restrictions.
Changes from v1.0	Adds explicit recommendation ranking, eligibility and empty-profile scenarios.

TC-PERF-01: Listing Search and API Load Performance

Field	Value
Requirement(s)	PERF-1, PERF-2

Field	Value
Type	Performance / Load
Preconditions	The application is deployed or running in a dedicated test environment. The database contains 5,000 listings. A load-testing tool and 50 concurrent virtual users are available.
Steps	1. Prepare the 5,000-listing test dataset. 2. Start the application and record the environment and commit SHA. 3. Configure 50 concurrent virtual users. 4. Exercise filtered listing search and record response-time percentiles. 5. Exercise representative non-upload API calls, including attempt, answer, result and analytics calls. 6. Record p50, p95, maximum response time and error rate separately for each scenario. 7. Record cold-cache and warm-cache behavior.
Test data	5,000 listings and representative student, attempt, answer, result and analytics data.
Expected result	Filtered listing search has p95 response time below 2 seconds. Other API calls, excluding file uploads, have p95 response time below 1 second under the same 50-user load. Record errors and environment details.
Actual result	Not run on the Issue #87 branch. Historical release evidence from 30 October 2026: search p95 37 ms and other API p95 62 ms with 50 virtual users. These are historical results, not a fresh execution.
Status	Partial — Render 07-11-2026 after the #123 fix: PERF-1 search p95 327 ms Pass; PERF-2 other calls p95 1,679 ms Fail (#123; 3,584 ms on 06-11). See performance-rerun.md
Automated by	`docs/testing/search_load.jmx (currently exercises listing searches and /api/health only; attempt, answer, result and analytics API samplers remain to be added or verified for complete PERF-2 load coverage)` – load-test plan. `server/tests/integration/listing-search-performance.test.js` provides a separate synthetic 5,000-listing service-level performance check; it is not a substitute for the 50-user load test.
Changes from v1.0	Explicitly documents PERF-1 and PERF-2 load conditions, percentile targets, API categories and historical-versus-current execution evidence.

PERF-3: Placement Dashboard Rendering for 2,000 Students

Field	Value
Requirement(s)	PERF-3
Type	Performance / End-to-end
Preconditions	A test environment contains the demo cohort of 2,000 students and the required analytics data. The client production build and API are available. An authorized user can access the placement dashboard.
Steps	1. Prepare the 2,000-student demo data set in an isolated test database. 2. Start the API and production client build. 3. Authenticate as an authorized dashboard user. 4. Open the placement dashboard and measure from navigation until the dashboard finishes rendering. 5. Record the authorized API request, computation and UI rendering time. 6. Repeat the measurement and record cold-cache and warm-cache behavior separately. 7. Check the dashboard's empty-data and API-error states separately.
Test data	Demo data generated for 2,000 students, associated profiles, applications and results.

Field	Value
Expected result	The complete dashboard renders in less than 3 seconds. Authorized data is displayed correctly. Empty-data and API-error states are handled clearly and do not cause an indefinite loading state. API latency and end-to-end browser rendering time are recorded separately.
Actual result	Not run on the Issue #87 branch. Historical release evidence: the 30 October 2026 test report records browser rendering of 0.19–0.29 seconds with 2,000 students. This is not a fresh execution for this issue.
Status	Blocked on Render (200 students seeded, 2,000 needed); Pass locally 30-10-2026 (0.19–0.29 s)
Automated by	`server/tests/integration/analytics-performance.test.js` covers the API performance portion. Browser rendering requires a client-side measurement; record the actual measurement method and environment when executed.
Changes from v1.0	Separates the full dashboard rendering requirement from API-only performance measurements and explicitly includes empty and error states.

4. Updated Search Test Case

TC-SRCH-01: Public Listing Search, Filters, Sorting and Pagination

Field	Value
Requirement(s)	SRCH-1, SRCH-2, SRCH-3
Type	Functional / Boundary / Negative
Preconditions	The API is running and contains the approved demo listing data. No authentication is required to browse or search public listings.
Steps	1. Send `GET /api/listings` without an authentication token. 2. Search with `search=<keyword>` matching a listing title. 3. Search by a company name and a skill keyword. 4. Filter by location and `mode=REMOTE`. 5. Apply `minStipend` and `maxStipend` together. 6. Apply `deadlineBefore` and `deadlineAfter`. 7. Apply a skill filter. 8. Sort with `sortBy=stipend&order=desc` (supported `sortBy`: `deadline`, `stipend`, `postedAt`, `title`; `order`: `asc` or `desc`). 9. Request subsequent pages and inspect pagination metadata. 10. Submit an invalid mode, invalid sort field and a stipend range where the minimum exceeds the maximum.
Test data	Public demo listings; title, company, location and skill keywords; valid mode; minimum and maximum stipend values; ISO date filters; supported sort fields; invalid mode and sort field; minimum stipend greater than maximum stipend.
Expected result	Guest requests can browse and search listings without authentication. Only open, unexpired listings are returned. Keyword search matches title, company and skills. Location, mode, skill, minimum stipend, maximum stipend and deadline filters narrow the results correctly. The response is `{listings, pagination}`; sorting returns the requested order and `pagination` has accurate `page`, `limit`, `total` and `totalPages`. Invalid values and an inverted stipend range return HTTP 400 validation errors.
Actual result	**Render, 04-11-2026 (#88, SVR 4.4):** search, mode, stipend and skill queries returned the expected results; stipend-descending order monotonic over 50 records; pages 1 and 2

Field	Value
	had no duplicate IDs; invalid mode, sort and stipend range gave 400. Newman: 10 requests, 15 assertions, 0 failures.
Status	Pass
Automated by	`server/tests/integration/listing-search.test.js` – public listing visibility, keyword, location, minimum stipend, skill, sorting, pagination, deadline filters, mode and validation. Guest browsing and positive `maxStipend` filtering are covered by this integration suite; the focused suite passed 13/13 tests.
Changes from v1.0	Adds explicit guest browsing, `maxStipend`, `deadlineBefore` and `deadlineAfter` coverage. Requires verified demo-seed counts rather than legacy expected counts.

5. UI Test Steps

These are manual UI checks. Do not describe them as automated unless a corresponding client test is implemented and executed.

UI-LST-01: Listing Search Page

1. Open the public listings/search page without signing in.
2. Confirm that listings are visible to a guest.
3. Search using a title or company keyword.
4. Apply location and work-mode filters.
5. Set minimum and maximum stipend values.
6. Apply a skill filter.
7. Apply deadline-before and deadline-after filters.
8. Apply the filters and inspect the result count and listing cards.
9. Change sorting and navigate to another result page.
10. Clear the filters and confirm the default listing results return.
11. Try an invalid stipend range and confirm the UI prevents or explains the invalid input.

Expected result: Guest browsing works, filters produce the expected results, pagination and sorting work, and invalid filter combinations are handled clearly.

UI-LST-02: Recruiter Listing Form

1. Sign in as an approved recruiter.
2. Open the listing creation form.
3. Enter valid company, title, location, mode, stipend, skills and deadline details.
4. Submit the form.
5. Confirm that the listing is created and appears in the recruiter's listings.
6. Open the listing for editing and change permitted fields.
7. Save the changes and confirm they are displayed.
8. Try invalid values, including a negative stipend, missing required fields and a past deadline.
9. Confirm that another recruiter cannot edit or close this listing.

Expected result: Valid listings can be created and edited by their owner. Invalid data is rejected with useful validation messages. Another recruiter cannot modify the listing.

Result (Render, 06-11-2026, Playwright/Edge, `recruiter01` / `recruiter02`): Pass. Empty submit showed six field errors; a negative stipend gave "Stipend must be 0 or more" and a past deadline "Deadline must be in the future". A valid listing was published, appeared in My listings, and an edited title was saved and shown. `recruiter02` got "You can only manage your own listings". Evidence: `../evidence/ui-render-results.json`, `../evidence/ui-render/lst02-*.png`.

UI-SRCH-04: Saved Internships

1. Sign in as a student.
2. Open the public listings page.
3. Save an eligible listing using its save control.
4. Open the saved-internships page.
5. Confirm that the listing appears once.
6. Return to the listing and remove it from saved internships.
7. Confirm that it disappears from the saved list.
8. Confirm that the saved count is displayed and that the 100-item limit is communicated when reached.

Expected result: Saved listings appear in the student's saved list, duplicate saves do not create duplicates, removal works and the limit is communicated clearly.

Result (Render, 06-11-2026, `student0003`): Pass. Saving from the listing page turned the control into "Remove from saved"; the Saved page listed it once with the count "3 of 100 saved"; removing it took it off the list. The 101st-save message was not reached on the shared data. Evidence: `../evidence/ui-render/srch04-*.png`.

UI-SRCH-05: Student Home Feed

1. Sign in as a student with a completed profile and skills.
2. Open the student home page.
3. Inspect the recommended internships section.
4. Confirm that recommendations display relevant matched skills.
5. Inspect upcoming deadlines.
6. Use the home-page search control to navigate to listing search.
7. Open a recommended listing and inspect its details.
8. Repeat with a student profile that has no skills.

Expected result: Recommendations are relevant to the student's profile, matched skills are visible, upcoming deadlines are displayed, search navigation works and an appropriate empty state appears when there are no recommendations.

Result (Render, 06-11-2026): Pass. `student0003`'s home showed "Recommended for you" cards with skill chips and a "Deadlines this week" panel; the home search for React opened `/listings?search=React`. A new student with no skills saw "No matches yet. Add skills and your CGPA on your profile to get recommendations." Evidence: `../evidence/ui-render/srch05-home.png`, `../evidence/ui-render/auth06-registered.png`.

6. Requirement-to-Test Case Mapping

Requirement ID	Test case / evidence
LST-1	TC-LST-01
LST-2	TC-LST-01
LST-3	LST-3
LST-4	TC-LST-01
SRCH-1	TC-SRCH-01
SRCH-2	TC-SRCH-01
SRCH-3	TC-SRCH-01
SRCH-4	SRCH-4
SRCH-5	SRCH-5

Requirement ID	Test case / evidence
PERF-1	TC-PERF-01 – listing search load performance
PERF-2	TC-PERF-01 – other API call performance
PERF-3	PERF-3 – end-to-end placement dashboard rendering

7. Execution Notes

1. Record the test date, environment, branch and commit SHA for each execution.
2. Record the Node.js version, database setup, client build mode and relevant configuration for performance runs.
3. Use an isolated test database for demo seeding. Do not run `seed:demo -- --reset` against a database containing personal, shared or otherwise important data.
4. Use `seed:demo` data for listing search expected counts. Record the exact seed command and the resulting counts alongside the test execution.
5. The listing-search performance integration test uses its own synthetic 5,000-listing fixture. Its expected counts must not be substituted for counts from `seed:demo`.
6. Record API response time separately from full browser dashboard rendering time.
7. Do not mark a test as Pass based only on the presence of an automated test in the repository. Execute the relevant test and record its result.
8. The 30 October 2026 release report records historical results from the `dev` working tree. Those results provide context but do not establish a new pass on the Issue #87 branch.
9. Guest browsing and positive `maxStipend` filtering are covered by `server/tests/integration/listing-search.test.js`; the focused suite passed 13/13 tests. Demo-seed count verification is complete; UI execution remains pending.

8. Changes Summary

- Updated the listing lifecycle and search test descriptions.
- Added explicit listing ownership and recruiter authorization coverage.
- Added saved-internship behavior, including the 100-bookmark limit.
- Added student recommendation ranking and eligibility checks.
- Added the 2,000-student placement dashboard rendering case for PERF-3.
- Added public guest browsing and the `maxStipend`, `deadlineBefore` and `deadlineAfter` search filters.
- Added UI test steps for listing search, recruiter listing forms, saved internships and the student home feed.
- Added requirement-to-test mapping for all LST, SRCH and PERF IDs.
- Left execution results pending until tests are run on the Issue #87 branch.

Test cases: applications, review and notifications (v2.0)

Owner: Guru R Bharadwaj (Member 3). Requirements: SRS 4.5 (APP-1 to APP-5), SRS 4.6 (REV-1 to REV-4) and DOM-1.

These are the Test Plan v1.0 section 13 cases, updated to match what was built, and new cases for requirements that v1.0 covered only through the automated suites. The changes rely on DEVIATIONS.md and the applications API.

How to run

- **Environment:** the Part-3 test build in deployment.md section 6. API base `https://oippp-api.onrender.com/api`, client `https://oippp-client.onrender.com`.
- **Accounts:** `recruiter01@example.com` (R1), `recruiter02@example.com` (R2), `student0001@students.example.com` (S1), `student0002@students.example.com` (S2). The password for all of them is `Demo12345`.
- **Resume:** before the run, S1 and S2 upload fixtures/resume.pdf on **My profile**. The seeded students have no resume.
- **Listings:** R1 creates these on the **Post a listing** form at the start of the run:

Name	Settings
L1	Open, no minimum CGPA, deadline in 30 days
L2	Open, deadline in 30 days, then closed by R1 with **Close** in **My listings** (or `PATCH /listings/:id` with `{"status":"CLOSED"}`)
L3	Open, minimum CGPA 8.0, deadline in 30 days

The seeded listings have random CGPA limits, so the run uses its own. - **Email:** the API sends to the team's Mailtrap sandbox inbox. Open the inbox before steps that check email. - **Recording:** fill in Actual result and Status for each case, with the environment, date and commit. The build commit is in deployment.md section 6.

"Automated by" names the Jest file and test under `server/tests/` that cover the same behaviour. All 486 server tests passed locally on 2 Nov 2026 at commit `f044d21`.

Summary

TC	Title	Requirements	v1.0 → v2.0
TC-APP-01	Apply once and record history	APP-1, APP-4	Updated
TC-APP-02	Reject applying to a closed listing or when not eligible	APP-2, APP-3, DOM-1	Updated
TC-APP-03	My Applications: status and history, newest first	APP-5	New
TC-APP-04	Eligibility boundary: CGPA equal to the minimum	APP-3, DOM-1	New
TC-REV-01	Status transitions and notification	REV-2, REV-3, REV-4	Updated
TC-REV-02	Recruiter sees applicants of own listings only	REV-1	New
TC-REV-03	Full lifecycle to Offer, and rejection with a reason	REV-2, REV-3, APP-4	New

TC	Title	Requirements	v1.0 → v2.0
TC-REV-04	Two recruiters' tabs change the same application at once	REV-2, APP-4	New
TC-REV-05	Notification bell: list and mark read	REV-4	New

TC-APP-01: Apply once and record history

Field	Value
Requirement(s)	APP-1, APP-4
Type	Functional
Preconditions	S1 has a resume and no application to L1. L1 is open.
Steps	**API:** 1. As S1, `POST /applications` with `{listingId: L1, coverNote}`. 2. Send the same request again. 3. `GET /applications/mine`. 4. As R1, open the bell or `GET /notifications`. **UI:** 1. As S1, open L1 from Search, write the cover note and press **Submit application**. 2. Reload L1. 3. Open **My applications** and press **History**.
Test data	L1; a 200-character cover note. For the boundary, a 1,000-character note is accepted and a 1,001-character note is refused.
Expected result	Step 1: 201 `{applicationId, status: "APPLIED"}`. Step 2: 409 `ALREADY_APPLIED`, "You have already applied to this internship", and `details` carries the existing `applicationId`. Step 3: one item for L1 whose history has one entry, `null → APPLIED`, with the actor and time. Step 4: R1 has an `APPLICATION_RECEIVED` notification, "New application for ". UI: after applying, the page says "You have applied to this internship" instead of showing the form, and the counter under the cover note shows `n/1000`. The application stores the resume, and **Resume sent** in My applications opens it. A 1,001-character note disables **Submit application** and the API returns 400.
Actual result	**API (Render, 02-11-2026, `f044d21`, SVR 4.2):** 7 of 7 checks pass: 201 `APPLIED`, repeat gives 409 `ALREADY_APPLIED` with the existing `applicationId`, one history entry `null → APPLIED`, R1 gets the notification, resume copy opens, 1,001-character note gives 400. **UI (Render, 06-11-2026, Playwright/Edge, `student0003` on a new listing):** cover note entered and **Submit application** pressed; after reload the form is no longer offered; the application is listed in **My applications** and **History** shows `Applied`. Evidence: `evidence/ui-render-results.json`, `evidence/ui-render/app01-*.png`.
Status	Pass (API and UI)
Automated by	`integration/apply.test.js`: "first apply is 201 APPLIED, second is 409, list shows one entry", "stores the current resume and notifies the recruiter in-app"; `integration/reliability.test.js`: "concurrent submits create exactly one application"
Changes from v1.0	Response body and the `ALREADY_APPLIED` details now match the API contract (DEVIATIONS row 18 for the shared error table). Added: recruiter notification, stored resume, the 1,000-character limit (APP-1) and UI steps.

TC-APP-02: Reject applying to a closed listing or when not eligible

Field	Value
Requirement(s)	APP-2, APP-3, DOM-1
Type	Functional (negative)
Preconditions	L2 is closed. L3 has minimum CGPA 8.0. S2's profile CGPA is set to 7.4 on **My profile**.
Steps	**API:** 1. As S2, `POST /applications` for L2. 2. As S2, apply to L3. 3. As R1, apply to L1. **UI:** 1. As S2, open L2. 2. Open L3 and press **Submit application**.
Test data	L2, L3; S2 with CGPA 7.4
Expected result	Step 1: 409 `LISTING_CLOSED`, "Applications for this internship are closed". Step 2: 422 `NOT_ELIGIBLE`, "Minimum CGPA required is 8.0". Step 3: 403, because only students can apply. UI: L2 shows Status **Closed** and the warning "Applications for this internship are closed." with no form. L3 shows the error banner naming 8.0. No application is created in either case.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** 4 of 4 checks pass: 409 `LISTING_CLOSED`, 422 `NOT_ELIGIBLE` naming 8.0, 403 for a recruiter, nothing stored. UI steps not run.
Status	Pass (API); UI not run
Automated by	`integration/apply.test.js`: "closed listing is 409 LISTING_CLOSED with the SRS message", "past deadline counts as closed", "CGPA below minimum is 422 naming 8.0", "recruiters cannot apply"
Changes from v1.0	Error code `NOT_ELIGIBLE` and the exact messages added. A listing past its deadline counts as closed even before the deadline job runs. DOM-1 is now traced here.

TC-APP-03: My Applications: status and history, newest first

Field	Value
Requirement(s)	APP-5
Type	Functional
Preconditions	S1 applied to L1 (TC-APP-01), then to L3, with CGPA 8.0 or more (TC-APP-04). R1 has shortlisted S1 on L1.
Steps	**API:** 1. As S1, `GET /applications/mine`. 2. As S2, `GET /applications/mine`. **UI:** 1. As S1, open **My applications**. 2. Press **History** on L1.
Test data	S1, S2
Expected result	Step 1: the L3 application comes before L1 (newest first). Each item has `listing` (title, company, status, deadline) and `history` newest first: L1 shows `APPLIED → SHORTLISTED`, then `null → APPLIED`. Step 2: S2 sees none of S1's applications. UI: the table shows Internship, Company, Applied and Status, and the History dialog lists each change with its time.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** 4 of 4 checks pass: newest first, history newest first, S2 sees only S2's applications. **UI (06-11-2026): My applications** table and **History** panel shown for `student0003` (see TC-APP-01 evidence).
Status	Pass (API and UI)

Field	Value
Automated by	`integration/apply.test.js`: "my applications are newest first", "a student can see only their own applications"
Changes from v1.0	New. v1.0 covered APP-5 only in the automated suite.

TC-APP-04: Eligibility boundary: CGPA equal to the minimum

Field	Value
Requirement(s)	APP-3, DOM-1
Type	Boundary
Preconditions	L3 has minimum CGPA 8.0.
Steps	1. Set S1's profile CGPA to 7.99 and apply to L3. 2. Set it to 8.0 and apply to L3 again.
Test data	CGPA 7.99, 8.0
Expected result	Step 1: 422 `NOT_ELIGIBLE`. Step 2: 201 `APPLIED`. A CGPA equal to the minimum is eligible.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** CGPA 7.99 gives 422 `NOT_ELIGIBLE`; 8.0 gives 201 `APPLIED`.
Status	Pass
Automated by	`integration/apply.test.js`: "eligible student can apply to the CGPA-gated listing" (above the minimum; the equal case is checked only here)
Changes from v1.0	New

TC-REV-01: Status transitions and notification

Field	Value
Requirement(s)	REV-2, REV-3, REV-4
Type	Functional
Preconditions	Application A1 (S1 on L1) is APPLIED. R1 owns L1. The Mailtrap inbox is open.
Steps	**API:** 1. As R1, `PATCH /applications/A1/status {status: "SHORTLISTED"}`. 2. `{status: "APPLIED"}`. 3. `{status: "REJECTED"}` with no `reasonCode`. 4. As S1, `GET /notifications`. Check the Mailtrap inbox. **UI:** 1. As R1, open **My listings → Applicants** for L1 and press **Move to Shortlisted** on S1. 2. Press **Reject**, leave the reason empty and confirm. 3. As S1, open the bell.
Test data	A1
Expected result	Step 1: 200 `{status: "SHORTLISTED"}`. Step 2: 422 `INVALID_TRANSITION`, "Cannot move an application from SHORTLISTED to APPLIED". Step 3: 400 `REASON_REQUIRED`, with `details` listing the five reason codes; A1 stays SHORTLISTED. Step 4: one `APPLICATION_STATUS` notification, ": Shortlisted". Within 2 minutes, one email to `student0001@students.example.com` with the same subject arrives in Mailtrap. Nothing is sent for the refused steps. UI: the reject dialog shows "Choose a reason" and does not submit.

Field	Value
Actual result	**API (Render, 02-11-2026, SVR 4.2):** 6 of 6 checks pass, including the shortlist email in Mailtrap 6 s after the change. **UI (Render, 06-11-2026, Playwright/Edge):** as `recruiter01`, **Applicants → Move to Shortlisted** changed the status to Shortlisted in the table. Evidence: `evidence/ui-render/rev01-shortlisted.png`.
Status	Pass (API and UI)
Automated by	`integration/status.test.js`: "shortlist 200, back to APPLIED 422 INVALID_TRANSITION, one notification + one email"; `integration/review.test.js`: "reject without a reason is 400 REASON_REQUIRED"; `integration/notifications.test.js`: "writes the in-app row and sends one email", "failed email keeps the row and retries 3 times (1, 5, 15 min), then FAILED", "a retry succeeds once mail is back"
Changes from v1.0	The email check now uses the Mailtrap sandbox (the v1.0 run used a local inbox). Error codes `INVALID_TRANSITION` and `REASON_REQUIRED` added. Retry rule: a failed email is retried after 1, 5 and 15 minutes, then marked FAILED, by the in-process job (DEVIATIONS row 4). Retries are proven by the Jest test only, because Mailtrap does not fail on request.

TC-REV-02: Recruiter sees applicants of own listings only

Field	Value
Requirement(s)	REV-1
Type	Functional, Security
Preconditions	S1 and S2 have applied to L1 (S2 can apply to L1 because it has no minimum CGPA).
Steps	**API:** 1. As R1, `GET /listings/L1/applications`. 2. Add `?status=SHORTLISTED`. 3. Add `?limit=51`. 4. As R1, `GET /applications/<id>/resume` using a `resumeUrl` from step 1. 5. As R2, repeat steps 1 and 4. **UI:** 1. As R1, open **Applicants** for L1, filter Status to Shortlisted, and press **Resume**.
Test data	L1, R1, R2
Expected result	Step 1: 200 `{items, total, page}`. Each item has the student's name, branch, graduation year, CGPA, skills, cover note, status and `resumeUrl`. Step 2: only shortlisted applicants. Step 3: 400 (the page size is at most 50). Step 4: 200 `application/pdf`, the resume sent with the application. Step 5: 403 `NOT_LISTING_OWNER` for both. UI: the table shows Student, Profile, Applied and Status, and the resume opens in the viewer.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** 6 of 6 checks pass: page shape, status filter, limit 51 gives 400, owner gets the PDF, R2 gets 403 `NOT_LISTING_OWNER`. Re-checked 06-11-2026: `recruiter02` gets 403 `NOT_LISTING_OWNER` on another recruiter's application resume (`evidence/identity-render-results-2.json`). UI filter step not run.
Status	Pass (API); UI not run
Automated by	`integration/review.test.js`: "owner sees applicants with profile summary and resume link, filterable by status", "another recruiter is refused", "page size is capped at 50"; `integration/resume.test.js`: "listing owner can download the application resume", "another recruiter cannot download the application resume"
Changes from v1.0	New. v1.0 covered REV-1 only in the automated suite. The resume route is `GET /applications/:id/resume` (DEVIATIONS row 16).

TC-REV-03: Full lifecycle to Offer, and rejection with a reason

Field	Value
Requirement(s)	REV-2, REV-3, APP-4
Type	Functional
Preconditions	A1 is SHORTLISTED (after TC-REV-01). A2 (S2 on L1) is APPLIED.
Steps	**API:** 1. As R1, `GET /applications/reject-reasons`. 2. Move A1 to INTERVIEW, then OFFER. 3. Try A1 → REJECTED and A1 → INTERVIEW. 4. Try A2 → INTERVIEW (skipping a stage). 5. A2 → REJECTED with `reasonCode: "NOT_A_CODE"`. 6. A2 → REJECTED with `reasonCode: "SKILLS_MISMATCH"`. 7. Try A2 → SHORTLISTED. 8. As S1 and S2, `GET /applications/mine`. **UI:** in Applicants, the buttons offered for each status are only the allowed next steps; **Reject** opens a dialog with the five reasons.
Test data	A1, A2
Expected result	Step 1: five reasons: `SKILLS_MISMATCH`, `POSITION_FILLED`, `INTERVIEW_NOT_CLEARED`, `INCOMPLETE_PROFILE`, `OTHER`, each with a label. Step 2: 200 twice. Step 3: 422 for both (OFFER is final). Step 4: 422. Step 5: 400. Step 6: 200; A2 stores `rejectReason`, and S2's notification and email end with "Reason: Skills do not match the role." Step 7: 422 (REJECTED is final). Step 8: A1's history is APPLIED → SHORTLISTED → INTERVIEW → OFFER, and A2's is APPLIED → REJECTED with the reason; every entry has `from`, `to`, `by` and `at`.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** 11 of 11 checks pass; the rejection email ends with "Reason: Skills do not match the role." and arrived on the third attempt (Mailtrap rate limit, app retry).
Status	Pass (API)
Automated by	`integration/status.test.js`: "skipping a stage is refused", "full path to OFFER is in history and the audit log; OFFER is final"; `integration/review.test.js`: "a reason outside the list is refused", "reject with a reason stores it and tells the student why", "the reason list is served to the client"; `unit/application-transitions.test.js`
Changes from v1.0	New. The reason list endpoint is an addition (DEVIATIONS row 1).

TC-REV-04: Two recruiters' tabs change the same application at once

Field	Value
Requirement(s)	REV-2, APP-4
Type	Reliability
Preconditions	A new application A3 (S2 on L3, after setting S2's CGPA to 8.0 or more) is APPLIED. R1 has a token.
Steps	Send two `PATCH /applications/A3/status {status: "SHORTLISTED"}` requests at the same moment, for example two `curl` commands started together with `&` in one shell. Then `GET /listings/L3/applications`.
Test data	A3

Field	Value
Expected result	One request returns 200. The other returns 409 `STALE_STATUS`, "The application changed meanwhile, please reload" if the two overlapped, or 422 `INVALID_TRANSITION` if it ran second. A3's history has exactly one `APPLIED → SHORTLISTED` entry and S2 gets one notification. Record which response the second request returned.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** two simultaneous shortlists gave one 200 and one 422 `INVALID_TRANSITION`; one history entry, one notification.
Status	Pass
Automated by	None; the conditional update is covered only by code review. Add a test if this case fails.
Changes from v1.0	New. `409 STALE_STATUS` is an addition (DEVIATIONS row 2).

TC-REV-05: Notification bell: list and mark read

Field	Value
Requirement(s)	REV-4
Type	Functional, UI
Preconditions	S1 has at least two unread notifications (after TC-REV-01 and TC-REV-03).
Steps	**API:** 1. As S1, `GET /notifications?unread=true`. 2. `PATCH /notifications/<id>/read` for one of them. 3. Repeat step 1. 4. As S2, `PATCH /notifications/<S1's id>/read`. 5. `GET /notifications` with no token. **UI:** 1. As S1, check the bell badge. 2. Open the bell and press **Mark read** on one item.
Test data	S1, S2
Expected result	Step 1: S1's unread notifications only, newest first, without the `email` delivery fields. Step 2: 200 with `read: true`. Step 3: one fewer. Step 4: 404, because users can only mark their own. Step 5: 401. UI: the badge count falls by one, and the item changes from unread to read without a reload.
Actual result	**API (Render, 02-11-2026, SVR 4.2):** 5 of 5 checks pass: unread list, mark read, other user's notification 404, no token 401. **UI (Render, 06-11-2026):** after the shortlist, `student0003`'s bell showed "Notifications, 1 unread" and the item "…: Shortlisted" with a **Mark read** link (not pressed in this run). Evidence: `evidence/ui-render/rev05-bell.png`.
Status	Pass (API); UI partial (badge and list checked, Mark read not pressed)
Automated by	`integration/notifications.test.js`: "lists only my own, filters unread, marks read, hides email state", "requires login"
Changes from v1.0	New. `PATCH /notifications/:id/read` is an addition (DEVIATIONS row 1).

Requirement coverage

Requirement	Test cases
APP-1	TC-APP-01

Requirement	Test cases
APP-2	TC-APP-02
APP-3	TC-APP-02, TC-APP-04
APP-4	TC-APP-01, TC-REV-03, TC-REV-04
APP-5	TC-APP-03
REV-1	TC-REV-02
REV-2	TC-REV-01, TC-REV-03, TC-REV-04
REV-3	TC-REV-01, TC-REV-03
REV-4	TC-REV-01, TC-REV-05
DOM-1	TC-APP-02, TC-APP-04

DOM-2 (one-offer policy) and SEC-6 (audit log) are in identity.md (#83).

Aptitude and Analytics Test Cases

Project: OIPPP
Version: Part-3 Testing (Test Case Document v2.0)
Owner: E S Sukruth (Member 4)
Environment: Render test build (deployment.md section 6)
Issue: #95 (update test cases); results from #96

1. Purpose and scope

These cases cover the aptitude test engine (`TST-1` to `TST-6`), progress tracking and placement analytics (`ANL-1` to `ANL-3`), the per-graduation-year reporting rule (`DOM-3`) and the safety requirements (`SAFE-1`, `SAFE-2`). They replace TC-TST-01, TC-TST-02 and TC-ANL-01 from Test Plan v1.0 section 13 and add a case for every requirement that v1.0 only covered in the automated suite.

Each case lists API steps (run with Postman, newman or `server/scripts/system-test-aptitude.js`) and, where the screen matters, UI steps on https://oippp-client.onrender.com. A status is filled in only after the case has been run on the test build, with the environment, date and commit.

Test data. `seed:demo` gives 5 published tests (*Placement Practice Test 1–5*, 20 questions, 30 minutes each), 100 questions, 200 students (`student0001` … `student0200@students.example.com`) and 10 recruiters, all with password `Demo12345`. Cases that need a known answer key or a short test create their own questions and tests with titles starting `ST-96`, so the seeded data is not changed. The admin password is shared privately and is not stored here.

Error format. Errors are `{ "error": { "code", "message" } }` (API contract).

2. Updated v1.0 test cases

TC-TST-01: Resume an attempt after refresh or a closed tab with no lost answers

Field	Value
Requirement(s)	TST-3, TST-4, SAFE-1
Type	Functional / Reliability
Preconditions	Student A is logged in and has no attempt on the chosen published test (20 questions, 30 minutes).
Steps	**API:** 1. `POST /tests/:id/attempts` (empty body). 2. Answer 5 questions with `PATCH /attempts/:id/answers` `{questionId, optionIdx}`, including `optionIdx: 0`, and wait for each response. 3. Change one answer. 4. Simulate a refresh: `GET /attempts/:id`. 5. Simulate a closed and reopened tab: `POST /tests/:id/attempts` again. 6. Send 3 starts at once. **UI:** 1. *Aptitude tests* → *Start or resume* on the test. 2. Pick 5 answers; wait for "✓ Saved" after each. 3. Press F5. 4. Close the tab, open the site again, *Aptitude tests* → *Start or resume*.
Test data	Student A; Placement Practice Test *n* with no attempt for A.
Expected result	Step 1 returns 201 with `attemptId`, `startedAt`, `endsAt` = start + 30 min, `serverTime`, 20 questions and no `answerIdx`. Every save returns 204 only after it is stored; the screen shows "Saving…" then "✓ Saved". Step 4 returns 200 with all 5 answers (the changed one with its new value) and the same `endsAt`. Step 5 returns **200** (not 201) with the same `attemptId` and `endsAt`. All 3 concurrent starts return 200 with the same attempt. In

Field	Value
	the UI the answers are restored after F5 and after reopening, and the timer continues from the original deadline (it does not restart at 30:00).
Actual result	API (05:28 UTC, `student0187`, `T1 mvzb6i0f`, evidence `docs/testing/evidence/aptitude-analytics-student-run.json`): start 201 with 20 questions, `endsAt` = `startedAt` + 30 min and no `answerIdx`; 5 saves 204 (including option 0); a changed answer 204; `GET /attempts/:id` returned all 5 answers with the changed value and the same `endsAt`, 1,787 s left; restart returned **200** with the same `attemptId` and `endsAt`; 3 concurrent starts all 200 with the same attempt. A second attempt (`student0188`, 20 saves) also returned all 20 answers after a refresh. UI: see UI-TST-01, UI-TST-02 and TC-SAFE-01.
Status	Pass (API) — Render, 05-11-2026, `main` @ `1e07391`; UI steps pass on the local build (UI-TST-01, UI-TST-02 and TC-SAFE-01)
Automated by	`server/tests/integration/aptitude-recovery.test.js` (refresh, reopening, serverTime); `aptitude-api.test.js` ("start and resume use one server deadline…", "each acknowledged save survives reload…", "simultaneous start creates exactly one attempt"); `aptitude-indexes.test.js`
Changes from v1.0	Saves are acknowledged (204) before the screen shows them as saved; reopening must return the same attempt and the same `endsAt`; adds option 0, a changed answer and concurrent starts. Deadline comes from the server (`endsAt – serverTime`), so a wrong device clock does not change it.

TC-TST-02: Auto-submit at end time and score by topic

Field	Value
Requirement(s)	TST-5, TST-6
Type	Functional / Boundary
Preconditions	Admin has created 6 `ST-96` questions with known keys (Quantitative 2, Logical 2, Technical 2) and published a 10-minute test from them. Student B has no attempt on it.
Steps	**API:** 1. Student B starts the test. 2. Answer 4 correctly, 1 wrongly, leave 1 unanswered. 3. About 2 s before `endsAt`, save the unanswered question. 4. Make no other calls until after `endsAt` (the browser is not needed for the timeout). 5. 5 s after `endsAt`, save an answer. 6. 40 s after `endsAt`, save an answer. 7. `GET /attempts/:id` and `GET /attempts/:id/result`. **UI:** start the test, answer, and leave the page open past 0:00 to see it move to the result.
Test data	6 `ST-96` questions with known `answerIdx`; 10-minute test.
Expected result	Step 3 returns 204. Steps 5 and 6 both return **409 `ATTEMPT_CLOSED`** (hard cutoff at `endsAt`, no 30-second grace). The attempt is `SUBMITTED` with `submissionReason: TIMEOUT`, submitted by the server's timeout sweep shortly after `endsAt`. The result is 4/6 = 66.67% with topics in the order Quantitative 2/2 (100%), Logical 2/2 (100%), Technical 0/2 (0%), and topic counts add up to the totals. In the UI the timer reaches 0:00 and the page opens the result.
Actual result	**Local (admin run, `ST-96` 10-minute test, 6 questions with known keys):** an answer ~2 s before `endsAt` 204; 5 s and 40 s after `endsAt` 409 `ATTEMPT_CLOSED`; with no client calls the server sweep set `SUBMITTED` / `TIMEOUT` **1.0 s after `endsAt`**; result 4/6 = 66.67%, Quantitative

Field	Value
	2/2, Logical 2/2, Technical 0/2. **Render:** Run with a seeded 30-minute test instead of the 10-minute `ST-96` test (that one needs the admin account). `student0188` saved 20 answers on *Placement Practice Test 2* at 05:31 UTC and then made no calls; `endsAt` 06:00:37.935Z. Checked at 14:31 UTC: attempt `SUBMITTED` / `TIMEOUT`, score 20/20 = 100% from the saved answers (each topic 5/5, counts add up), and a later save returned 409 `ATTEMPT_CLOSED`. **But** `submittedAt` was 14:31:09, the time of the check, not shortly after `endsAt`: the free Render service had gone to sleep, so the one-second sweep was not running, and the attempt was closed by the read-time expiry check (DEVIATIONS row 11). No answer was accepted after the deadline and no time was added. The 5 s / 40 s late-answer probes and the exact topic split need the admin run. Evidence: `docs/testing/evidence/aptitude-timeout-render.json`.
Status	Pass on the local build — Local build (in-memory MongoDB replica set, `seed:demo` data), 05-11-2026, `dev` @ `8cb16a0`; evidence `docs/testing/evidence/aptitude-analytics-local-admin-run.json` (89 of 89 checks pass). Render: Partial — hard cutoff and TIMEOUT scoring pass; server-side auto-submit at `endsAt` not observed because the free instance slept; Render, 05-11-2026, `main` @ `1e07391`
Automated by	`server/tests/unit/attempt-timeout-job.test.js`; `server/tests/integration/aptitude-api.test.js` ("server sweep catches overdue attempts without a client…", "GET recovery finalizes overdue state", 0–30 s window); `aptitude-recovery.test.js` ("reconnecting finds a timed-out attempt…"); `unit/aptitude-scoring.test.js`
Changes from v1.0	Hard cutoff: **any** answer at or after `endsAt` returns 409 `ATTEMPT_CLOSED`, not only answers 40 s late (DEVIATIONS row 10). Auto-submit is done by the server's one-second timeout sweep, and the Result is created in the same transaction that closes the attempt (row 11). Test shortened to 10 minutes (the TST-2 minimum) with 6 questions so the expected per-topic numbers are exact.

TC-ANL-01: Placement statistics and CSV export

Field	Value
Requirement(s)	ANL-2, ANL-3
Type	Functional / Security
Preconditions	Admin logged in. `seed:demo` data loaded (200 students with branch and graduation year, 600 applications, 199 results).
Steps	**API:** 1. `GET /analytics/placements` (whole batch). 2. `GET /analytics/placements?branch=<branch>&gradYear=<year>`. 3. `GET /analytics/placements/export` with the same filter. 4. Repeat 1 and 3 as a student and as a recruiter. **UI:** *Placement statistics*, enter the branch and graduation year, *Apply*, read the tiles and charts, then *Export CSV* and open the file.
Test data	A branch and graduation year present in the seed, for example `CSE` and `2027`.
Expected result	Step 1 returns 200 with `students`, `placedStudents` (distinct students with at least one OFFER), `offersPerCompany`, `applicationsPerListing` and `averageScoreByBranch`; placed students ≤ total offers, and offers per company add up to offers per listing. Step 2 narrows every metric to the same cohort, echoes the filter and is no larger than step 1. Step 3 returns 200 `text/csv` with `attachment; filename="placement-report.csv"`; its `students` and `placedStudents` rows equal step 2. Step 4 returns 403 for both roles. In the

Field	Value
	UI the placement rate equals placed ÷ students for the filter, and the CSV numbers equal the screen. No combined readiness score appears anywhere.
Actual result	**Local (admin run):** whole batch 200 in 53 ms: 200 students, 99 placed, 99 offers both per company and per listing, branch averages CSE 65.3, EEE 64.0; `CSE` + 2027 → 26 students, 17 placed; CSV 200 `text/csv` attachment whose `students` (26) and `placedStudents` (17) rows equal the filtered dashboard; recruiter export 403. The UI check is UI-ANL-02. **Render:** Partial: student and recruiter got 403 on `/analytics/placements` and student 403 on `/export`. Dashboard and CSV comparison need the admin account.
Status	Pass on the local build — Local build (in-memory MongoDB replica set, `seed:demo` data), 05-11-2026, `dev` @ `8cb16a0`; evidence `docs/testing/evidence/aptitude-analytics-local-admin-run.json` (89 of 89 checks pass). Render: Not run (admin part); role checks pass — Render, 05-11-2026, `main` @ `1e07391`
Automated by	`server/tests/integration/analytics.test.js` (whole batch, filters, empty cohort, CSV quoting and formula escaping, cache); `analytics-performance.test.js` (2,000 students, API part of PERF-3)
Changes from v1.0	Routes are `GET /analytics/placements` and `GET /analytics/placements/export` (DEVIATIONS row 21). There is no combined readiness score (row 13). The v1.0 fixture (40 CSE 2027 students, 12 with offers) is not in `seed:demo`, so the check compares the dashboard, the filter and the CSV against each other instead of fixed numbers. The CSV is one `section,group,detail,value` table, not one row per student.

3. New test cases

TC-TST-03: Question bank create, edit and archive

Field	Value
Requirement(s)	TST-1
Type	Functional / Boundary / Negative / Security
Preconditions	Admin logged in. A student and a recruiter account are available.
Steps	**API:** 1. `POST /questions` with 2 options and with 6 options. 2. `POST /questions` with 1 option, 7 options, `answerIdx` out of range, fractional `answerIdx`, unknown topic `History`, unknown difficulty and duplicate options. 3. `PATCH /questions/:id` with the current `version` and new text. 4. Repeat step 3 with the old `version`. 5. `PATCH` an `answerIdx` that is outside the current options. 6. `PATCH {version, archived: true}`. 7. `GET /questions` and `GET /questions?archived=true`. 8. As a student and as a recruiter: `GET /questions` and `POST /questions`. 9. As a student, start a test and inspect the questions. **UI:** *Question bank* → *New question*, add options with *Add option*, mark the correct one, save; *Edit* it; archive it and switch the *Show* filter between *Active* and *Archived*.
Test data	Question texts prefixed `ST-96`; topics Quantitative, Logical, Verbal, Technical; difficulties Easy, Medium, Hard.
Expected result	Step 1 returns 201 for both, with `answerIdx` in the admin DTO. Every request in step 2 returns 400. Step 3 returns 200 with the new text and a new `version`. Step 4 returns 409 `STALE_VERSION`. Step 5 returns 400. Step 6 returns 200 with `archived: true`. Step 7 lists the

Field	Value
	question only under `archived=true`. Step 8 returns 403. Step 9: student question objects contain only `questionId, text, topic, difficulty, options`, never `answerIdx`.
Actual result	**Local (admin run):** 2- and 6-option questions 201 with `answerIdx` in the admin DTO; 1 option, 7 options, out-of-range and fractional `answerIdx`, unknown topic and difficulty, duplicate options all 400; edit with current version 200, stale 409 `STALE_VERSION`, `answerIdx` outside the options 400; archive 200 and listed only under `archived=true`. **Render:** Partial: student and recruiter got 403 on `GET /questions`, student 403 on `POST /questions`; student attempt questions had only `questionId, text, topic, difficulty, options`. Admin steps 1–7 need the admin account (run `system-test-aptitude.js --admin`).
Status	Pass on the local build — Local build (in-memory MongoDB replica set, `seed:demo` data), 05-11-2026, `dev` @ `8cb16a0`; evidence `docs/testing/evidence/aptitude-analytics-local-admin-run.json` (89 of 89 checks pass). Render: Not run (admin part); role checks pass — Render, 05-11-2026, `main` @ `1e07391`
Automated by	`server/tests/integration/aptitude-api.test.js` ("admin creates a question…", "bank filters, bounds and archive…", "editing validates answer against changed options and rejects stale versions", "invalid id and unauthorized test creation…"); `server/tests/unit/aptitude-models.test.js`
Changes from v1.0	New. Adds the edit and archive routes, version conflicts and answer-key redaction (DEVIATIONS row 9).

TC-TST-04: Test draft, publish and archive

Field	Value
Requirement(s)	TST-2
Type	Functional / Boundary / Negative
Preconditions	Admin logged in; at least 6 active `ST-96` questions and one archived question exist.
Steps	**API:** 1. `POST /tests` with 4 questions, with duration 9 and 121, with a duplicate question ID, and with an archived question. 2. `POST /tests` with 5 questions and 120 minutes. 3. As a student, try to start the draft. 4. `PATCH` the draft's `questionIds` to 6 questions. 5. Publish with the old `version`. 6. Publish with the current `version`. 7. `PATCH` the published test's title. 8. As a student, `GET /tests`. 9. Archive it. 10. Try to publish it again. 11. As a student, `GET /tests`. **UI:** *Tests → Build a test*: enter a title and *Duration (minutes)*, tick questions, save; *Publish* (confirm the dialog); *Archive*.
Test data	6 `ST-96` questions, 1 archived question.
Expected result	Step 1 returns 400 each time (`INVALID_QUESTIONS` for the archived question). Step 2 returns 201 with status `DRAFT`. Step 3 returns 404. Step 4 returns 200 with 6 questions. Step 5 returns 409 `STALE_VERSION`. Step 6 returns 200 `PUBLISHED`. Step 7 returns 409 `TEST_LOCKED`. Step 8 lists the test. Step 9 returns 200 `ARCHIVED`. Step 10 returns 409. Step 11 no longer lists the test. In the UI the *Publish* and *Archive* buttons ask for confirmation and a published test can no longer be edited.
Actual result	**Local (admin run):** 4 questions, 9 and 121 minutes, duplicate IDs 400; an archived question 400 `INVALID_QUESTIONS`; 5 questions / 120 minutes 201 `DRAFT`; student start of the draft 404; draft edit 200; stale publish 409 `STALE_VERSION`; publish 200; edit after

Field	Value
	publish 409 `TEST_LOCKED`; listed for students; archive 200; republish 409 `TEST_LOCKED`; hidden from students. **Render:** Partial: student `POST /tests` 403; the student list showed only `PUBLISHED` tests (6). Admin steps need the admin account.
Status	Pass on the local build — Local build (in-memory MongoDB replica set, `seed:demo` data), 05-11-2026, `dev` @ `8cb16a0`; evidence `docs/testing/evidence/aptitude-analytics-local-admin-run.json` (89 of 89 checks pass). Render: Not run (admin part); role checks pass — Render, 05-11-2026, `main` @ `1e07391`
Automated by	`server/tests/integration/aptitude-api.test.js` ("create draft, edit, publish, list and archive without exposing keys", "duplicate, missing and archived questions cannot be used", "a concurrent document version error is reported as a stale edit", "draft/archived tests and non-students cannot start")
Changes from v1.0	New. Adds the draft → published → archived life cycle and version conflicts (DEVIATIONS row 9).

TC-TST-05: Only one submitted attempt per test

Field	Value
Requirement(s)	TST-3, TST-6
Type	Functional / Negative / Security
Preconditions	Student A has an in-progress attempt (from TC-TST-01). Student B is logged in.
Steps	**API:** 1. Student B: `GET /attempts/:id`, `PATCH /attempts/:id/answers` and `POST /attempts/:id/submit` on A's attempt. 2. Student A: `GET /attempts/:id/result` before submitting. 3. `POST /attempts/:id/submit`. 4. Submit again. 5. `GET /attempts/:id/result`; also as student B. 6. Save another answer. 7. `POST /tests/:id/attempts` again. 8. `GET /tests`. **UI:** *Submit* (the dialog warns about unanswered questions), then go back to *Aptitude tests*.
Test data	Student A and B; the attempt from TC-TST-01.
Expected result	Step 1 returns 404 for all three (no data about A's attempt). Step 2 returns 409 `ATTEMPT_OPEN`. Step 3 returns 200 with `score`, `correctAnswers`, `totalQuestions` and `topicBreakdown`, whose counts add up to the totals and whose `score` = 100 × correct ÷ total. Step 4 returns the same `resultId`. Step 5 returns the same result for A and 404 for B. Step 6 returns 409 `ATTEMPT_CLOSED`. Step 7 returns 409 `ATTEMPT_EXISTS`. Step 8 shows `myAttempt.status: SUBMITTED`. In the UI the test shows *View result* instead of *Start or resume*.
Actual result	API: B got 404 on read, save and submit of A's attempt; result before submit 409 `ATTEMPT_OPEN`; submit 200, 4/20 = 20%, topics Quantitative 2/5, Logical 0/5, Verbal 1/5, Technical 1/5 (sums match totals); repeat submit returned the same `resultId`; `GET result` matched, B got 404; attempt `SUBMITTED` / `MANUAL`; a later save 409 `ATTEMPT_CLOSED`; restart 409 `ATTEMPT_EXISTS`; test list showed `myAttempt.status: SUBMITTED`. UI: see UI-TST-01 and UI-TST-03.
Status	Pass (API) — Render, 05-11-2026, `main` @ `1e07391`; UI steps pass on the local build (UI-TST-01 and UI-TST-03)

Field	Value
Automated by	`server/tests/integration/aptitude-api.test.js` ("other students cannot read, save, submit or fetch a result", "manual submit is idempotent, terminal…", "students see their own attempt on each listed test", "an open attempt has no result yet"); `aptitude-indexes.test.js` ("result indexes prevent duplicate scoring records")
Changes from v1.0	New. Separates single-attempt enforcement and owner isolation from the resume case.

TC-TST-06: Answer save acknowledged within 1 second

Field	Value
Requirement(s)	TST-4
Type	Performance / Reliability
Preconditions	Student logged in with a fresh attempt on a 20-question test.
Steps	**API:** 1. For each of the 20 questions, call `GET /health` (network baseline) and then `PATCH /attempts/:id/answers`, timing both from the client. 2. Save with `optionIdx` = number of options. 3. Retry a stored answer with an old `version` (lost 204). 4. Send a different answer with an old `version`. **UI:** pick answers quickly and watch the save status under the question.
Test data	Placement Practice Test with no attempt for the student.
Expected result	Every save returns 204 within 1,000 ms of being sent, measured at the client. Step 2 returns 400 `INVALID_ANSWER`. Step 3 returns 204 without a conflict. Step 4 returns 409 `STALE_VERSION`. In the UI "Saving…" changes to "✓ Saved" within a second, and a failed save shows "Not saved".
Actual result	20 saves from a team laptop to Render (Singapore): median 931 ms, p95 1,023 ms, max 1,191 ms; 16 of 20 under 1 s. In the first run 1 of 5 saves took 2,832 ms. Network baseline in the same run (`GET /health`, no database): median 694 ms, p95 990 ms, so about 240 ms of each save is server and database time. Out-of-range option 400 `INVALID_ANSWER`; lost-204 retry 204; stale different answer 409 `STALE_VERSION`. Evidence: `docs/testing/evidence/aptitude-save-latency.json`.
Status	Fail (bug #118) — 4 of 20 saves over 1 s at the client; Render, 05-11-2026, `main` @ `1e07391`
Automated by	`server/tests/integration/aptitude-recovery.test.js` ("retrying a save whose 204 was lost…", "a different answer from a stale screen is still a conflict"); `aptitude-api.test.js` ("stale client versions and invalid snapshot answers are rejected"); client `client/src/test/pages.test.jsx` ("restores saved answers and auto-saves a new pick")
Changes from v1.0	New. v1.0 had no timing check for TST-4. The time is measured at the client, so the network round trip to Render counts; the `/health` baseline separates network time from server time.

TC-ANL-02: Student progress view

Field	Value
Requirement(s)	ANL-1

Field	Value
Type	Functional / Security
Preconditions	Student A has just submitted a test (TC-TST-05). Another seeded student has no results.
Steps	**API:** 1. Before and after A's submission, `GET /analytics/me`. 2. Compare `applications.byStatus` with `GET /applications/mine`. 3. `GET /analytics/me` for the student with no results. 4. `GET /analytics/me` as a recruiter. **UI:** *My progress*: read *Tests taken*, *Average score*, the *Score trend* chart, *Weakest topics*, *All topics* and *Applications by status*.
Test data	Student A; a student with no results (for example `student0200`).
Expected result	After submitting, `testsTaken` is one higher and the new result is the last point of `trend` (oldest first, with `testTitle`, `score`, `submittedAt`). `averageScore` is the mean of the trend scores. `weakestTopics` is the first two entries of `topics`, sorted weakest first. `byStatus` lists all five statuses and matches the applications list; `total` matches its length. The student with no results gets `averageScore: null` and empty `trend`, `topics` and `weakestTopics` (not 0%). A recruiter gets 403. The UI shows the same numbers and, for a student with no tests, "Take a test to see your trend." and "No topic data yet." instead of 0%. No combined readiness score is shown.
Actual result	API: after A's submit `testsTaken` went 1 → 2 and the new result was the last trend point (score 20); `averageScore` 35 = mean of the trend; `weakestTopics` = first two of `topics` (Logical 10.0%, Technical 30.0%); `byStatus` {APPLIED 1, SHORTLISTED 1, INTERVIEW 0, OFFER 0, REJECTED 1}, total 3, matching `GET /applications/mine`; `student0200` (no results) got `averageScore: null` and empty lists; recruiter 403. UI: see UI-ANL-01.
Status	Pass (API) — Render, 05-11-2026, `main` @ `1e07391`; UI steps pass on the local build (UI-ANL-01)
Automated by	`server/tests/integration/analytics.test.js` ("a student with no data gets empty states…", "trend, pooled weakest topics and application counts use only the caller's records", "one assessed topic yields one weak topic…", "student readiness is never cached"); client `client/src/test/pages.test.jsx` ("shows progress figures and weakest topics")
Changes from v1.0	New. Covers `GET /analytics/me` and the decision not to compute a readiness score (DEVIATIONS rows 13, 21). Weak topics are pooled across all results (decision D2).

TC-DOM-03: Statistics per graduation year

Field	Value
Requirement(s)	DOM-3, ANL-2
Type	Functional / Boundary
Preconditions	Admin logged in; seed data loaded.
Steps	**API:** 1. `GET /analytics/placements?gradYear=y` for each year 2025–2028. 2. `GET /analytics/placements?branch=<b>&gradYear=2027` and the same with the branch in lower case. 3. `gradYear=2099` (no students). 4. `gradYear=1999`. **UI:** *Placement statistics* → *Graduation year* `2027` → *Apply*; then enter `1999`.
Test data	Years 2025–2028, 2099 and 1999.

Field	Value
Expected result	Each year returns 200 with a cohort no larger than the whole batch, and the filter echoed in `filters`. The branch filter ignores case. 2099 returns `students: 0` and empty lists (the CSV adds "No students match the selected cohort"). 1999 returns 400 `VALIDATION`. In the UI a year outside 2000–2100 shows "Year must be 2000–2100" and the request is not sent.
Actual result	**Local (admin run):** 2025: 0, 2026: 0, 2027: 104, 2028: 96 students, each within the batch of 200; `CSE`/`cse` + 2027 both 26; 2099 → 0 students and empty lists; 1999 → 400 `VALIDATION`. UI: 1999 shows "Year must be 2000–2100" (UI-ANL-02).
Status	Pass on the local build — Local build (in-memory MongoDB replica set, `seed:demo` data), 05-11-2026, `dev` @ `8cb16a0`; evidence `docs/testing/evidence/aptitude-analytics-local-admin-run.json` (89 of 89 checks pass). Not run on Render (needs the admin account)
Automated by	`server/tests/integration/analytics.test.js` ("branch (case-insensitive) and gradYear filters apply to every metric", "an empty cohort returns zero students and empty lists", "an empty cohort says so instead of showing bare zeroes")
Changes from v1.0	New. v1.0 did not list DOM-3. The cohort is the student's current Profile branch and graduation year (decision D4).

TC-SAFE-01: Refresh, closed tab and dropped network in a real browser

Field	Value
Requirement(s)	SAFE-1, TST-4, TST-5
Type	Reliability / UI
Preconditions	Student logged in on the deployed client in Chrome, Firefox or Edge, with an in-progress attempt.
Steps	1. Pick 3 answers; wait for "✓ Saved". 2. Press F5. 3. Pick 2 more answers, close the tab, reopen the site and click *Start or resume*. 4. Open DevTools → Network → *Offline*, pick an answer, then go back *Online*. 5. Note the timer before and after each step.
Test data	Any published test without a submitted attempt for the student.
Expected result	After F5 and after reopening, every answer that showed "✓ Saved" is still selected and the timer keeps counting down from the same deadline. While offline the new answer shows "Not saved" and is not shown as saved; after reconnecting it can be saved again. No answer is lost and no extra time is given.
Actual result	Local, built-in browser: 3 answers picked (each "✓ Saved"); F5 restored all 3 and the timer went 29:35 → 29:32 over 3 s (no restart). The tab was closed, a new tab opened and *Start or resume* returned the same attempt with 3 answers and the timer at 29:03. Network loss was simulated by failing every request in the page (the pane has no offline switch): the status stayed "Saving…" during the client's 3 retries, then showed "Not saved" and the banner "Cannot reach the server. Check your connection and try again."; after the network came back, the next pick showed "✓ Saved". At 360 × 740 there was no horizontal scroll and the timer and *Submit* stayed visible. Firefox, Edge and a real phone are in #97.

Field	Value
Status	Pass (Chromium only) — Local build (Windows 11, in-memory MongoDB replica set, Vite dev server, `seed:demo` data), Chromium-based built-in browser of the Claude desktop app, 05-11-2026, `dev` @ `8cb16a0`
Automated by	`server/tests/integration/aptitude-recovery.test.js` (server side); `client/src/test/timer.test.js` (countdown from `serverTime`)
Changes from v1.0	New. Moves the browser part of SAFE-1 out of TC-TST-01 so it can be run on each browser for QA-5 (#97).

TC-SAFE-02: Deactivation keeps the profile, attempts and results

Field	Value
Requirement(s)	SAFE-2, PROF-4
Type	Functional / Reliability / Security
Preconditions	Admin logged in. A seeded student with at least one result and one application is chosen for the test (deactivation cannot be undone through the API, so use one student only, for example `student0199`).
Steps	**API:** 1. As the student, note `GET /analytics/me` and `GET /applications/mine`; note the admin `GET /analytics/placements` totals. 2. Admin: `PATCH /admin/users/:id/deactivate`. 3. Call `GET /analytics/me` with the student's old token. 4. Log in as the student. 5. Admin: `GET /admin/users?search=<email>` and `GET /admin/users/:id/resume`. 6. After the 5-minute statistics cache expires, `GET /analytics/placements` again. **UI:** *Users* → search the student → *Deactivate*; then try to log in as that student.
Test data	One seeded student.
Expected result	Step 2 returns 200. Steps 3 and 4 are refused (401/403). Step 5 shows the user with status `DISABLED` and the resume is still available (or 404 if the student never uploaded one). Step 6 still counts the student's results and applications: totals do not drop. Nothing is deleted.
Actual result	**Local (admin run, `student0199`, 1 result and 3 applications):** deactivate 200; the old token 401 `UNAUTHENTICATED`; login 403 `FORBIDDEN`; admin user search shows `DISABLED`; profile resume route 404 (none uploaded), not an error; after the 5-minute cache, batch totals unchanged: results 202 → 202, applications 600 → 600, students 200 → 200. Nothing deleted. UI *Deactivate* button not exercised (the API call is the same).
Status	Pass on the local build — Local build (in-memory MongoDB replica set, `seed:demo` data), 05-11-2026, `dev` @ `8cb16a0`; evidence `docs/testing/evidence/aptitude-analytics-local-admin-run.json` (89 of 89 checks pass). Not run on Render (deactivation is irreversible on the shared data)
Automated by	`server/tests/integration/identity-edge-cases.test.js` ("deactivation keeps the profile and resume but blocks the account (SAFE-2)"); `identity-flow.test.js` ("disabling an account revokes an already-issued token"); `rbac.test.js`
Changes from v1.0	New. v1.0 left SAFE-2 for Sprint 3 review.

4. UI test cases

These check the screens on the deployed client. Run them in Chrome at desktop width first; #97 repeats the student ones on Firefox, Edge and a 360 px Android phone.

UI-TST-01: Aptitude tests list

Field	Value
Requirement(s)	TST-3, QA-5
Type	UI
Preconditions	Student logged in; at least one test not started, one in progress and one submitted.
Steps	1. Open *Aptitude tests*. 2. Read each card. 3. Click *Start or resume*, go back, and click it again. 4. Click *View result* on a submitted test.
Test data	Seeded tests and the attempts from TC-TST-01/05.
Expected result	The page says "Each test can be taken once. The timer runs on the server." Each card shows the title, "*n* questions" and "*n* minutes". A test not yet submitted shows *Start or resume*, and clicking it twice opens the same attempt with the timer still running; a submitted one shows *View result*. Draft and archived tests are not listed.
Actual result	Local: the list showed the six published tests with "*n* questions" and "*n* minutes"; draft and archived tests were not listed. *Start or resume* opened the attempt, and clicking it again from a new tab opened the same attempt with the timer still running. After submitting, the card showed "Score 15%" and *View result*.
Status	Pass — Local build (Windows 11, in-memory MongoDB replica set, Vite dev server, `seed:demo` data), Chromium-based built-in browser of the Claude desktop app, 05-11-2026, `dev` @ `8cb16a0`
Automated by	Not automated (client coverage of `Tests.jsx` is 0%, see the Branch Coverage Report)
Changes from v1.0	New

UI-TST-02: Test-taking screen and timer

Field	Value
Requirement(s)	TST-4, TST-5, SAFE-1, QA-5
Type	UI
Preconditions	Student has an in-progress attempt.
Steps	1. Open the attempt. 2. Use *Previous*, *Next* and the question palette. 3. Pick answers. 4. Wait until less than 5 minutes are left (or use the 10-minute `ST-96` test). 5. Click *Submit* with questions unanswered.
Test data	Any in-progress attempt.
Expected result	The timer bar ("Time left mm:ss") stays visible while scrolling. The palette marks each question *Answered* / *Not answered* in text, not only by colour. The save status shows "Saving…", then "✓ Saved". Under 5 minutes the bar turns to a warning and shows

Field	Value
	"Less than 5 minutes left. Answers are saved as you pick them." *Submit* asks "*n* question(s) are unanswered. Submit anyway?" before submitting.
Actual result	Local: the timer bar ("Time left mm:ss") is `position: sticky` and stayed on screen after scrolling, also at 360 px. The palette labels each question "answered" / "not answered" in text. Each pick showed "Saving…" then "✓ Saved" within about a second. *Submit* asked "15 question(s) are unanswered. Submit anyway?". The bar turned to a warning (`timer-bar warning`, `aria-live="assertive"`) with "Less than 5 minutes left. Answers are saved as you pick them." at 04:07 left on the 10-minute test, and at 0:00 the page moved to the result by itself (0/6, nothing answered).
Automated by	`client/src/test/pages.test.jsx` ("restores saved answers and auto-saves a new pick"), `client/src/test/timer.test.js`
Changes from v1.0	New

UI-TST-03: Result page

Field	Value
Requirement(s)	TST-6
Type	UI
Preconditions	Student has a submitted attempt.
Steps	1. Open *View result*. 2. Compare with `GET /attempts/:id/result`. 3. Click *Back to tests*.
Test data	Attempt from TC-TST-05.
Expected result	*Test result* shows the score and the *By topic* table (Topic, Correct, Score) with the same numbers as the API. No answer key is shown.
Actual result	Local: *Test result* showed 15% and 3/20, and the *By topic* table (Quantitative 1/5 20%, Logical 1/5 20%, Verbal 1/5 20%, Technical 0/5 0%) matched `GET /attempts/:id/result` exactly. No answer key was shown; *Back to tests* links to `/tests`.
Status	Pass — Local build (Windows 11, in-memory MongoDB replica set, Vite dev server, `seed:demo` data), Chromium-based built-in browser of the Claude desktop app, 05-11-2026, `dev` @ `8cb16a0`
Automated by	Not automated (`Result.jsx` 0% branch coverage)
Changes from v1.0	New

UI-TST-04: Admin question bank and test builder

Field	Value
Requirement(s)	TST-1, TST-2
Type	UI
Preconditions	Admin logged in.

Field	Value
Steps	1. *Question bank* → *New question*; try to save with no text, an empty option and two identical options; then save a valid one. 2. *Tests* → *Build a test*; try 4 questions and 9 minutes; then save 5 questions / 10 minutes. 3. *Publish*, then *Archive*.
Test data	`ST-96` questions.
Expected result	Invalid input shows a field message ("Question text is required", "Options cannot be empty", "Options must be different") and nothing is saved. Valid questions and tests appear in their lists. The status column changes Draft → Published → Archived, each after a confirmation dialog ("… This cannot be undone.").
Actual result	Local: *New question* with blank text and options showed "Question text is required" and "Options cannot be empty"; two identical options showed "Options must be different"; a valid question saved and appeared in the bank. *Build a test* with 4 questions and 9 minutes showed "Duration must be 10 to 120 minutes" and "Pick 5 to 60 questions (4 picked)" and saved nothing; 5 questions / 10 minutes saved as Draft. *Publish* and *Archive* each asked "… This cannot be undone." and the status moved Draft → Published → Archived (only *Archive* offered once published).
Status	Pass — Local build (Windows 11, in-memory MongoDB replica set, Vite dev server, `seed:demo` data), Chromium-based built-in browser of the Claude desktop app, 05-11-2026, `dev` @ `8cb16a0`
Automated by	Not automated (`Questions.jsx` 17%, `TestsAdmin.jsx` 16% client branch coverage)
Changes from v1.0	New

UI-ANL-01: Student progress dashboard

Field	Value
Requirement(s)	ANL-1, PERF-3
Type	UI
Preconditions	One student with results and applications; one without results.
Steps	1. Open *My progress* as each student. 2. Compare the numbers with `GET /analytics/me`. 3. Time how long the page takes to show the numbers (after the server is awake).
Test data	TC-ANL-02 students.
Expected result	Tiles, chart and tables match the API. The student with no tests sees the empty messages, not 0%. The page shows its data within 3 s.
Actual result	Local: *My progress* showed Tests taken 1, Average score 15%, Weakest topics Technical 0% (0/5) and Quantitative 20% (1/5), all four topics, and Applications by status (Applied 1, Shortlisted 1, Interview 0, Offer 0, Rejected 1), matching `GET /analytics/me`. Data appeared 234 ms after opening the page (360 px, no horizontal scroll). Empty-state wording was checked through the API on Render (TC-ANL-02), not on screen.
Status	Pass — Local build (Windows 11, in-memory MongoDB replica set, Vite dev server, `seed:demo` data), Chromium-based built-in browser of the Claude desktop app, 05-11-2026, `dev` @ `8cb16a0`

Field	Value
Automated by	`client/src/test/pages.test.jsx` ("shows progress figures and weakest topics")
Changes from v1.0	New

UI-ANL-02: Admin placement statistics dashboard

Field	Value
Requirement(s)	ANL-2, ANL-3, DOM-3, PERF-3
Type	UI
Preconditions	Admin logged in.
Steps	1. Open *Placement statistics* without filters. 2. Filter by branch and graduation year → *Apply*. 3. *Export CSV* and open the file. 4. Time how long the dashboard takes to show the numbers.
Test data	Seed data.
Expected result	The page shows *Students*, *With an offer*, *Placement rate*, *Offers per company*, *Applications per listing* and *Average test score by branch*. Filtering changes every section. The CSV `placement-report.csv` has the same numbers as the screen. The dashboard shows its data within 3 s.
Actual result	Local: *Placement statistics* showed Students 200, With an offer 99, Placement rate 49.5%, matching the API; data appeared 744 ms after opening. Graduation year 1999 showed "Year must be 2000–2100" and disabled *Apply*. Branch `cse` + 2027 → 26 students, 17 with an offer, 65.4%, the same as the API and the CSV. *Export CSV* requested `/analytics/placements/export?branch=cse&gradYear=2027` and saved `placement-report.csv`. All three chart sections rendered.
Status	Pass — Local build (Windows 11, in-memory MongoDB replica set, Vite dev server, `seed:demo` data), Chromium-based built-in browser of the Claude desktop app, 05-11-2026, `dev` @ `8cb16a0`
Automated by	Not automated (`Statistics.jsx` 0% branch coverage)
Changes from v1.0	New

5. Requirement coverage

Requirement	Test cases
TST-1	TC-TST-03, UI-TST-04
TST-2	TC-TST-04, UI-TST-04
TST-3	TC-TST-01, TC-TST-05, UI-TST-01
TST-4	TC-TST-01, TC-TST-06, TC-SAFE-01, UI-TST-02
TST-5	TC-TST-02, TC-SAFE-01, UI-TST-02
TST-6	TC-TST-02, TC-TST-05, UI-TST-03

Requirement	Test cases
ANL-1	TC-ANL-02, UI-ANL-01
ANL-2	TC-ANL-01, TC-DOM-03, UI-ANL-02
ANL-3	TC-ANL-01, UI-ANL-02
DOM-3	TC-DOM-03, UI-ANL-02
SAFE-1	TC-TST-01, TC-SAFE-01, UI-TST-02
SAFE-2	TC-SAFE-02
PERF-3 (UI part)	UI-ANL-01, UI-ANL-02

Every TST, ANL and SAFE requirement has at least one case. QA-1 (UAT) and QA-5 (browsers and devices) are measured in #97 and reported in the System Validation Report.

6. Summary

Group	Cases
Updated v1.0 cases	3
New API and browser cases	8
UI cases	6
Total	**17**